



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas para Toma de Decisión Artificial en Escenarios Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos limitados
bajo incertidumbre utilizando como escenario de prueba una crisis
pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis presenta el entorno de experimentación **mPES** (*Multiple Pandemic Experiment Suite*), un entorno multi-paquete para estudiar aprendizaje por refuerzo en la asignación secuencial de recursos bajo incertidumbre. El escenario de pandemia funciona como banco de pruebas controlado: cada agente debe administrar un presupuesto limitado mientras la severidad evoluciona a lo largo de la secuencia.

El estado actual del estudio compara doce modelos agrupados en dos suites: seis agentes individuales optimizados (`pes_ql`, `pes_dql`, `pes_dqn`, `pes_rdn`, `pes_a2c` y `pes_trf`) y seis variantes de ensamble, incluyendo `pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`. El paquete histórico `pes_base` aparece entre los artefactos individuales como referencia, pero no se incluye en este ranking comparativo. El análisis cubre 22 escenarios, una condición base y 21 perturbaciones, con 64 secuencias por celda y una semilla fija (42). La métrica principal es el rendimiento normalizado registrado como `global_mean_perf`.

Los resultados sitúan a `pes_trf` como el mejor modelo individual: alcanza $\bar{r} = 0,927$ en la condición de referencia y 0,930 como media en los 21 escenarios de perturbación, con una degradación media de $-0,002$ y la menor dispersión entre secuencias. Entre los ensambles, `pes_ens` obtiene la mejor media bajo perturbación (0,939) y la menor degradación máxima (0,037). La evidencia respalda la hipótesis de que el Transformer es el modelo individual que mejor generaliza en este entorno, pero no una superioridad del Transformer frente a todos los ensambles. La confianza de cada agente se cuantifica mediante $1 - H_{\text{norm}}$, con H_{norm} la entropía normalizada de la distribución derivada de sus valores de acción; la medida se registra por secuencia para todos los modelos individuales y actúa como factor de ponderación de los miembros en la decisión conjunta en los ensambles, constituyendo un indicador algorítmico y no una validación fisiológica. La sintonización de hiperparámetros emplea Optuna/TPE, mientras que el benchmark final se ejecuta en inferencia sobre artefactos ya entrenados.

Palabras clave: mPES, Aprendizaje por Refuerzo, Transformer Causal, *Deep Ensembles*, Cuantificación de Incertidumbre, Entropía de Shannon, Optimización Bayesiana.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del Problema	2
1.3. Hipótesis de Trabajo	2
1.4. Contribuciones	2
1.5. Estructura del Documento	3
2. Marco Teórico	3
2.1. Procesos de Decisión de Markov	4
2.2. Q-Learning tabular y aprendizaje basado en valores	4
2.2.1. Double Q-Learning	5
2.2.2. Modelado de recompensa mediante potencial (PBRS)	5
2.3. Deep Q-Networks (DQN)	5
2.3.1. Double DQN	6
2.4. Redes recurrentes y LSTM	6
2.5. Políticas, Gradientes y Actor-Critic	7
2.6. Atención y Transformer	7
2.7. Ensamble de modelos	8
2.8. Cuantificación de Incertidumbre y Entropía de Shannon	9
2.9. Optimización Bayesiana de Hiperparámetros	10
2.10. Distribución de entrenamiento y evaluación de la generalización	10
2.11. Contraste estadístico entre agentes	11
3. Estado de la Cuestión	12
3.1. Fundamentos neurocognitivos de la confianza algorítmica	12
3.2. Aprendizaje por Refuerzo en la Gestión de Crisis	13
3.3. Modelado secuencial mediante arquitecturas Transformer	14
3.4. Explicabilidad y cuantificación de incertidumbre	14
3.5. Generalización bajo perturbaciones controladas	14
3.6. Posición de este trabajo	15

4. Materiales y Métodos	15
4.1. Repositorio y reproducibilidad	16
4.2. Especificación del entorno <i>Pandemic Scenario</i>	16
4.2.1. Definición formal	16
4.2.2. Dinámica y recompensa	17
4.2.3. Estructura del benchmark	17
4.3. Modelos algorítmicos	18
4.3.1. Familia tabular	18
4.3.1.1. <code>pes_base</code>	19
4.3.1.2. <code>pes_q1</code>	19
4.3.1.3. <code>pes_dq1</code>	19
4.3.2. Familia profunda	20
4.3.2.1. Representación del estado.	20
4.3.2.2. Efecto sobre la comparación entre familias.	21
4.3.2.3. Configuración compartida.	21
4.3.2.4. <code>pes_dqn</code>	22
4.3.2.5. <code>pes_rdqn</code>	22
4.3.2.6. <code>pes_a2c</code>	23
4.3.2.7. <code>pes_trf</code>	23
4.3.3. Ensamblajes	24
4.3.3.1. <code>pes_ens.</code>	24
4.3.3.2. Variantes de combinación.	25
4.4. Procedimiento de optimización bayesiana	27
4.5. Referencia de comportamiento aleatorio	27
4.6. Variables internas del agente Transformer	28
4.7. Catálogo de escenarios de perturbación	29
4.8. Protocolo de entrenamiento y evaluación	31
5. Resultados	32
5.1. Familias de perturbaciones evaluadas	33
5.2. Suite individual: resumen cuantitativo	33
5.3. Reducción de la severidad normalizada	34
5.4. Comportamiento por familia algorítmica	35

5.5. Artefactos generados por el procedimiento experimental	36
5.6. Resultados por modelo	39
5.7. Resultados por escenario	43
5.8. Evaluación en escenarios de extrapolación	43
5.9. Suite de ensambles	44
5.10. Análisis de cuantificación de incertidumbre	50
6. Discusión	51
6.1. Modelos individuales	51
6.1.1. Interpretación del rendimiento del Transformer	52
6.2. Ensamblados y confianza en la decisión conjunta	53
6.3. Entropía como indicador de confianza algorítmica	54
6.4. Escenarios de extrapolación	55
6.5. Lectura de los resultados de <code>pes_a2c</code>	55
7. Conclusiones	55
7.1. Contraste de las hipótesis	56
7.2. Conclusiones principales	56
7.3. Limitaciones del estudio	57
7.4. Trabajo futuro	58
7.5. Conclusión general	59
8. Agradecimientos	59
Referencias Bibliográficas	60
Apéndice	63
A. Comandos de reproducción	63
B. Procedimiento de ejecución del benchmark	63
C. Escenarios de extrapolación	64

Índice de figuras

4.1. Severidad cruda del decisor aleatorio	29
4.2. Variables internas del Transformer	29
5.1. Desempeño medio por modelo y escenario (individual)	37
5.2. Degradación frente a la referencia (individual)	37
5.3. Significancia de Welch por escenario (individual)	37
5.4. Divergencia KL de acciones por escenario (individual)	38
5.5. Tamaño de efecto pareado entre modelos individuales	38
5.6. Resultados del modelo base	39
5.7. Resultados de Q-Learning optimizado	40
5.8. Resultados de Double Q-Learning	40
5.9. Resultados de DQN	41
5.10. Resultados de RDQN	41
5.11. Resultados de A2C	42
5.12. Resultados del Transformer	42
5.13. Curvas por secuencia y familia de perturbación	43
5.14. Ordenamiento por desempeño medio (individual)	44
5.15. Curvas en los escenarios de extrapolación	44
5.16. Ordenamiento de los ensambles	46
5.17. Curvas de los ensambles por familia de perturbación	46
5.18. Curvas de los ensambles en escenarios de extrapolación	47
5.19. Desempeño y degradación de los ensambles	47
5.20. Mapas estadísticos de los ensambles	48
5.21. Contrastes pareados de los ensambles	49
5.22. Resultados del ensemble	50

Índice de cuadros

4.1. Arquitecturas evaluadas	18
4.2. Catálogo de escenarios de perturbación	31
5.1. Desempeño normalizado medio por modelo	34
5.2. Comparaciones entre modelos individuales	35
5.3. Resultados de los ensambles bajo perturbación	45
5.4. Comparaciones entre ensambles	47
A.1. Comandos de reproducción	63

1. Introducción

La toma de decisiones complejas y secuenciales bajo incertidumbre constituye un desafío central para los sistemas de inteligencia artificial. En estos problemas, cada acción modifica el estado del entorno y condiciona las decisiones posteriores, por lo que una política eficaz debe anticipar consecuencias acumulativas y asignar recursos limitados a lo largo del tiempo. El entorno de experimentación **mPES** surge como una plataforma para comparar sistemáticamente arquitecturas de Aprendizaje por Refuerzo para este tipo de escenarios. La crisis pandémica se utiliza como banco de pruebas ya que es un caso concreto, exigente y reproducible que permite estudiar la adaptabilidad de los agentes frente a dinámicas no lineales, información incierta y condiciones de evaluación distintas a las del entrenamiento. Para ello, mPES extiende el entorno original *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1].

1.1. Motivación

La crisis sanitaria provocada por el COVID-19 ofrece un caso representativo de este tipo de escenarios. Gobiernos, hospitales y cadenas logísticas debieron ponderar beneficios inmediatos contra costos a largo plazo, mientras que cada asignación alteraría la evolución posterior del sistema sanitario. En este trabajo, la pandemia no se considera el límite del problema, sino el escenario experimental que permite observar y medir esas propiedades. La investigación busca así evaluar qué modelos matemáticos y computacionales producen decisiones con mayor adaptabilidad en escenarios secuenciales complejos.

El enfoque metodológico de esta tesis sigue la disciplina propia de la ingeniería: trazabilidad de cada artefacto, reproducibilidad estricta, control de versiones del entorno computacional y evaluación bajo cambios de condiciones como requisito de aceptación. Bajo esta óptica, mPES no se concibe como un experimento aislado sino como un *sistema* — doce modelos comparativos en dos suites, además del paquete histórico **pes_base** como referencia individual. El esquema de evaluación común y el procedimiento de ejecución reproducible están pensados para estudiar arquitecturas en dominios de alta complejidad.

1.2. Definición del Problema

En una crisis pandémica, un tomador de decisiones (un agente de RL) debe distribuir un *presupuesto limitado* de recursos médicos o logísticos entre diversas ciudades afectadas simultáneamente. Tres rasgos cualitativos definen el escenario: la *dinámica acumulativa* (la severidad no atendida crece de forma exponencial en los pasos subsiguientes), la *intervención temprana* (los recursos asignados generan un impacto sostenido que premia a los modelos capaces de anticiparla) y una *regla de evolución* lineal con saturación en cero:

$$S'_i = \max(0, \beta S_i - \alpha a_i) \quad \alpha = 0,4 \quad \beta = 1 + \alpha = 1,4$$

Donde S_i es la severidad de la ciudad i antes de la decisión, $a_i \in \{0, \dots, 10\}$ los recursos asignados, α la eficacia de cada unidad de recurso y β el factor de crecimiento por paso. La formulación completa como MDP (*Markov Decision Process*) finito y el *benchmark* fijo de 64 secuencias compartido por todos los enfoques se detallan en la Sec. 4.

1.3. Hipótesis de Trabajo

El trabajo se organiza alrededor de una hipótesis falsable desdoblada en dos niveles:

H1a (arquitectónica). *Entre los modelos individuales evaluados, el Transformer causal alcanza el mayor desempeño medio y conserva el rendimiento bajo cambios de severidad y longitud, utilizando la asignación de recursos durante una pandemia como escenario de prueba.*

H1b (confianza y decisión conjunta). *La cantidad $1 - H_{\text{norm}}$, donde H_{norm} es la entropía normalizada de la distribución de acciones de un agente, proporciona un indicador operativo de su confianza y puede emplearse como criterio de ponderación en la decisión conjunta de un ensamble de modelos, de manera análoga a la confianza estudiada en los sistemas de decisión conjunta humano-máquina.*

1.4. Contribuciones

Las principales contribuciones de esta tesis son:

1. Un *benchmark* reutilizable — *workspace* **mPES** — donde seis modelos individuales y seis variantes de ensamble comparten el mismo entorno Gymnasium [2], el mismo *pipeline* de optimización bayesiana con Optuna y el mismo esquema de evaluación sobre $n = 64$ secuencias *fijas*.
2. Una comparación cuantitativa y estadísticamente fundamentada entre métodos tabulares (`pes_q1`, `pes_dq1`), arquitecturas profundas (`pes_dqn`, `pes_rdqn`, `pes_a2c`, `pes_trf`) y seis ensambles (`pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`), con `pes_base` como referencia tabular sin optimización.
3. La integración de la **entropía de Shannon** como medida de confianza de la decisión: registrada por secuencia para cada agente individual y empleada como factor de ponderación de los miembros en los seis ensambles, planteada por analogía con las teorías neurocognitivas de la confianza humana.
4. Una implementación reproducible para Windows, con Python 3.12, TensorFlow 2.21 y artefactos de evaluación organizados por suite.

1.5. Estructura del Documento

El resto del documento se organiza de la siguiente forma. La Sección 2 introduce el marco teórico (MDPs, Q-Learning, redes recurrentes, atención, ensambles, entropía de Shannon). La Sección 3 presenta el estado de la cuestión (antecedentes neurocognitivos de la confianza, RL en gestión de crisis, Transformers en RL y cuantificación de incertidumbre). La Sección 4 describe la metodología: el escenario pandémico, las doce variantes evaluadas y el procedimiento experimental. La Sección 5 reporta los resultados obtenidos. La Sección 6 interpreta esos resultados a la luz de las hipótesis y la Sección 7 cierra el trabajo.

2. Marco Teórico

Este capítulo introduce el marco formal utilizado a lo largo de la tesis: procesos de decisión de Markov (MDP), aprendizaje por refuerzo basado en valor y en gradiente de política, aproximación de funciones con redes neuronales, codificadores recurrentes y basados en atención, optimización bayesiana de hiperparámetros y la entropía de Shannon como

medida de incerteza en la toma de decisión. El tratamiento es compacto; la referencia general es el texto de Sutton y Barto [3].

2.1. Procesos de Decisión de Markov

Un MDP finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ donde $\mathcal{S}$ es un espacio de estados finito, $\mathcal{A}$ un espacio de acciones finito, $P(s' | s, a)$ un núcleo de transición, $R(s, a, s')$ una función de recompensa acotada y $\gamma \in [0, 1)$ el factor de descuento. Bajo una política $\pi(a | s)$ el agente genera una trayectoria $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ cuyo *retorno descontado* desde el tiempo t es:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.1)$$

Donde r_{t+k+1} es la recompensa recibida $k+1$ pasos después de t . La función de valor de estado-acción de la política es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ y la política óptima satisface la ecuación de Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (2.2)$$

Donde la esperanza se toma sobre el estado siguiente $s' \sim P(\cdot | s, a)$ y la recompensa $r = R(s, a, s')$.

2.2. Q-Learning tabular y aprendizaje basado en valores

Cuando $\mathcal{S}$ y $\mathcal{A}$ son enumerables, Q se almacena como una tabla y se aplica la actualización clásica de Watkins [4]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.3)$$

Con tasa de aprendizaje $\alpha \in (0, 1]$ y selección de acciones ε -greedy; con probabilidad ε se elige una acción uniforme al azar y, en caso contrario, $\arg \max_a Q(s, a)$.

2.2.1. Double Q-Learning

El operador máx en la Ecuación (2.3) introduce un sesgo positivo en la estimación del valor. *Double Q-Learning* [5] mantiene dos tablas Q^A y Q^B actualizadas alternativamente, desacoplando la selección y la evaluación de la acción:

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', \arg \max_{a'} Q^A(s', a')) - Q^A(s, a)] \quad (2.4)$$

Y de forma simétrica para Q^B , eligiendo en cada paso con probabilidad 1/2 cuál de las dos tablas se actualiza.

2.2.2. Modelado de recompensa mediante potencial (PBRs)

El *Potential-Based Reward Shaping* (PBRs) [6] añade a la recompensa un término $F(s, s') = \gamma \Phi(s') - \Phi(s)$, donde $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ es una función de potencial sobre los estados. Ng *et al.* demostraron que esta forma de *shaping* preserva el conjunto de políticas óptimas del MDP original. En mPES se utiliza con $\Phi(s) = -S$ (severidad actual con signo negativo) y un coeficiente multiplicativo κ , de modo que la recompensa efectiva es $\tilde{r} = r + \kappa F(s, s')$.

2.3. Deep Q-Networks (DQN)

Cuando el espacio de estados es grande, $Q(s, a; \theta)$ se aproxima con una red neuronal de parámetros θ . El *Deep Q-Network* [7] estabiliza el entrenamiento con dos ingredientes: (i) un *buffer* de repetición de experiencia (*experience replay*) [8] del cual se muestrean pequeños lotes de transiciones en forma uniforme, rompiendo la correlación temporal entre muestras consecutivas, y (ii) una *red objetivo* (*target network*) $Q(\cdot; \theta^-)$ cuyos parámetros se copian desde θ cada C pasos. La función de costo es:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.5)$$

Donde $\mathcal{D}$ es el *buffer* de transiciones almacenadas. La red objetivo desacopla el blanco de la regresión del parámetro que se optimiza y se actualiza por copia cada C pasos. En

mPES la pérdida se optimiza con Adam [9] y se aplica recorte de la norma global del gradiente para limitar actualizaciones excesivas.

2.3.1. Double DQN

El operador $\max_{a'}$ sobre la red objetivo en la Ecuación (2.5) hereda el sesgo de sobreestimación del Q-Learning tabular. *Double DQN* [10] traslada la idea de la Ecuación (2.4) al caso profundo sin introducir una segunda red: la acción de *bootstrap* se selecciona con la red en línea θ y se evalúa con la red objetivo θ^- , que ya forma parte del esquema DQN:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) \quad (2.6)$$

A diferencia del Double Q-Learning tabular, no hay dos estimadores simétricos actualizados en alternancia, la red objetivo es una copia retrasada de θ y cumple el papel de evaluador. Todos los agentes de la familia DQN de mPES (`pes_dqn`, `pes_rdqn` y `pes_trf`) utilizan el blanco de la Ecuación (2.6) en lugar del de la Ecuación (2.5), con las acciones no factibles enmascaradas antes del $\arg \max$, y sustituyen el error cuadrático por la pérdida de Huber [11].

2.4. Redes recurrentes y LSTM

Cuando el estado observado no es markoviano — por ejemplo, cuando la dinámica de severidad acumula información histórica no representada en S_t — es habitual recurrir a codificadores recurrentes. La *Long Short-Term Memory* (LSTM) [12] introduce compuertas de entrada i_t , olvido f_t y salida o_t , cada una con valores en $(0, 1)$ obtenidos por una sigmoide sobre la entrada actual y el estado oculto previo, que controlan cómo el estado de celda c_t acumula información a largo plazo:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad h_t = o_t \odot \tanh(c_t) \quad (2.7)$$

Donde $\tilde{c}_t$ es el candidato de celda calculado con una tangente hiperbólica, h_t el estado oculto y $\odot$ el producto elemento a elemento. Hausknecht y Stone [13] mostraron que

incorporar una capa LSTM dentro de un DQN (*Deep Recurrent Q-Network*, DRQN) mejora el desempeño en entornos parcialmente observables. El paquete `pes_rdqn` adopta esta estrategia.

2.5. Políticas, Gradientes y Actor–Critic

Los métodos de *gradiente de política* [14] parametrizan la política $\pi_\theta(a \mid s)$ y actualizan θ a lo largo del gradiente del retorno esperado $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) A^{\pi_\theta}(s, a)] \quad (2.8)$$

Con la función de *ventaja* $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$, donde $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$. El *Advantage Actor–Critic* (A2C) [15] entrena simultáneamente un actor que produce π_θ y un crítico que estima V_ϕ . La ventaja se estima con *Generalized Advantage Estimation* (GAE):

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k} \quad \delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.9)$$

Donde δ_t es el error de diferencia temporal de un paso y $\lambda \in [0, 1]$ interpola entre la estimación de un paso ($\lambda = 0$, menor varianza y mayor sesgo) y la de Monte Carlo ($\lambda = 1$). El actor se regulariza con un término de entropía $-\beta_H H(\pi_\theta(\cdot \mid s))$, con $\beta_H \geq 0$, que desincentiva la concentración prematura de la política. En mPES la entropía se calcula sobre la política restringida a las acciones factibles en s y renormalizada, de modo que el término no premia la dispersión hacia acciones que el entorno no admite; el valor de β_H se fija por optimización bayesiana en un rango que incluye el cero, y el mejor ensayo resultó en $\beta_H > 0$. El estudio empírico de Andrychowicz *et al.* [16] resume las decisiones de diseño que más inciden en este tipo de métodos.

2.6. Atención y Transformer

El *Transformer* [17] reemplaza la recurrencia por *atención de producto escalar escalado*. Dadas las matrices de consultas Q , claves K y valores V , obtenidas por proyecciones lineales de la secuencia de entrada, y siendo d_k la dimensión de las claves,

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.10)$$

En un entorno secuencial, una máscara triangular M con $M_{ij} = -\infty$ para $j > i$ y $M_{ij} = 0$ en otro caso impide que la posición i atienda a posiciones futuras:

$$\text{Atención}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \quad (2.11)$$

En RL, Parisotto *et al.* [18] y Chen *et al.* [19] muestran que los codificadores Transformer pueden reemplazar a las redes recurrentes en tareas que requieren memoria, con capacidad para capturar dependencias entre posiciones distantes de la secuencia. En esta tesis cada bloque Transformer incluye conexiones residuales y normalización de capa, como en la formulación original.

2.7. Ensamble de modelos

Un *ensemble* combina las salidas de K modelos ya entrenados para producir una única decisión, con el objetivo de obtener un comportamiento de mayor estabilidad que el de sus miembros por separado. Sea m_k el k -ésimo miembro y $p_k(a \mid s)$ la distribución de probabilidad sobre las acciones que se deriva de su salida; cuando el miembro produce valores $Q_k(s, \cdot)$ en lugar de probabilidades, la distribución se obtiene con una *softmax* de temperatura τ ,

$$p_k(a \mid s) = \frac{e^{Q_k(s, a)/\tau}}{\sum_{a'} e^{Q_k(s, a')/\tau}} \quad (2.12)$$

La regla de combinación más habitual es el *voto suave* (*soft voting*); un promedio ponderado de las distribuciones:

$$p_{\text{ens}}(a \mid s) = \frac{\sum_{k=1}^K w_k p_k(a \mid s)}{\sum_{k=1}^K w_k} \quad w_k \geq 0, \quad (2.13)$$

Del que la acción final es $\arg \max_a p_{\text{ens}}(a \mid s)$. La alternativa es el *voto duro* (*hard voting*), en el que cada miembro emite un único voto por su acción preferida $a_k = \arg \max_a p_k(a \mid s)$

y se elige la más votada

$$a_{\text{ens}} = \arg \max_a \sum_{k=1}^K w_k \mathbf{1}[a_k = a] \quad (2.14)$$

Donde $\mathbf{1}[\cdot]$ es la función indicadora. El voto suave conserva la incertidumbre de cada miembro y el voto duro sólo su decisión. Los pesos w_k pueden ser fijos o depender de una medida de confianza $c_k \in [0, 1]$ del miembro; en esta tesis la confianza se define como $c_k = 1 - H_{\text{norm}}(p_k)$ (Ecuación (2.16)), de modo que un miembro con una distribución más concentrada pesa más en la combinación. La formulación de *deep ensembles* de Lakshminarayanan *et al.* [20] muestra que promediar varias redes es un procedimiento simple para estimar la incertidumbre predictiva sin modificar el entrenamiento de los miembros. Los ensambles de esta tesis son de *sola inferencia*: reutilizan artefactos ya entrenados y no introducen parámetros nuevos, por lo que las diferencias entre variantes se deben únicamente a la regla de combinación. Las seis reglas concretas se detallan en la Sec. 4.

2.8. Cuantificación de Incertidumbre y Entropía de Shannon

Dada una distribución de probabilidad discreta $p(a \mid s)$ sobre $|\mathcal{A}|$ acciones, la *entropía de Shannon* se define como:

$$H(p) = - \sum_{a \in \mathcal{A}} p(a \mid s) \log p(a \mid s) \quad (2.15)$$

Con la convención $0 \log 0 = 0$. Su valor máximo, $\log |\mathcal{A}|$, se alcanza con la distribución uniforme. La versión normalizada es:

$$H_{\text{norm}}(p) = \frac{H(p)}{\log |\mathcal{A}|} \in [0, 1] \quad (2.16)$$

La cual es independiente de la base del logaritmo y se interpreta como una medida de dispersión en la toma de decisión: $H_{\text{norm}} \rightarrow 0$ indica que la masa de probabilidad

se concentra en una acción; $H_{\text{norm}} \rightarrow 1$ indica indiferencia entre acciones. En esta tesis se utiliza $1 - H_{\text{norm}}$ como indicador de confianza del agente, por analogía con los marcadores de confianza estudiados en decisión humana por Boldt y Yeung [21] y revisados por Fleming [22]. Cuando la salida del modelo son valores Q y no probabilidades, la implementación de mPES los desplaza a valores no negativos y los normaliza antes de aplicar la Ecuación (2.15); el resultado es, por tanto, un indicador heurístico y no una probabilidad calibrada.

2.9. Optimización Bayesiana de Hiperparámetros

Un agente de RL tiene varios hiperparámetros no diferenciables (α, γ , tamaño del *buffer* de repetición, esquema de decaimiento de ε , anchura de red, entre otros) cuyas interacciones a su vez tampoco son lineales. La *optimización bayesiana* trata el puntaje de validación $y = f(x)$ de una configuración x como una función de caja negra ruidosa y la explora con un modelo probabilístico auxiliar. Se utiliza el estimador estructurado en árbol de Parzen (TPE) [23] implementado en Optuna [24]. Dado un umbral y^* , fijado como un cuantil de los puntajes observados, TPE modela por separado las densidades $\ell(x) = p(x \mid y < y^*)$ y $g(x) = p(x \mid y \geq y^*)$ mediante kernels estimadores de densidad (KDE). Para una muestra unidimensional $x_1, \dots, x_n$, la estimación es:

$$\hat{p}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) \quad (2.17)$$

Donde K es el kernel (función núcleo), y $h > 0$ el ancho de banda, que controla el grado de suavizado. Como el objetivo es maximizar el puntaje, TPE propone nuevas configuraciones donde el cociente $g(x)/\ell(x)$ es alto, es decir, donde la densidad de configuraciones buenas supera a la de configuraciones malas.

2.10. Distribución de entrenamiento y evaluación de la generalización

Sea $P_{\text{train}}(s, a, r, s')$ la distribución de transiciones observadas durante el entrenamiento y sea $P_{\text{test}}(s, a, r, s')$ la distribución utilizada para evaluar la política aprendida. La condición

de referencia conserva las mismas leyes de generación que el entrenamiento, de modo que:

$$P_{\text{test}} = P_{\text{train}} \quad (2.18)$$

Una evaluación bajo perturbación modifica una o más propiedades de esa distribución, por lo que $P_{\text{test}} \neq P_{\text{train}}$. En mPES el cambio se introduce en la distribución de severidades iniciales, en la de longitudes de secuencia o en ambas; la regla de transición (Ecuación (4.2)) y la función de recompensa permanecen fijas. El rendimiento medido bajo estas perturbaciones permite examinar la capacidad de la política para mantener su desempeño cuando cambian las condiciones de entrada, y no solamente su ajuste a las condiciones conocidas durante el entrenamiento.

2.11. Contraste estadístico entre agentes

Para comparar dos muestras de rendimiento A y B , cada una con n_A y n_B observaciones (en mPES, los valores por secuencia de $\bar{r}$), se reportan tres cantidades:

- **Tamaño del efecto:** la d de Cohen [25]:

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p} \quad s_p = \sqrt{\frac{(n_A - 1) s_A^2 + (n_B - 1) s_B^2}{n_A + n_B - 2}} \quad (2.19)$$

donde $\bar{x}$ y s^2 son la media y la varianza muestral de cada grupo y s_p la desviación estándar combinada. Como referencia descriptiva, $|d| \approx 0,2, 0,5$ y $0,8$ suelen interpretarse como efectos pequeño, medio y grande.

- **Significancia:** la prueba t de Welch [26] contrasta la igualdad de medias sin asumir varianzas iguales, donde el estadístico es:

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_A^2/n_A + s_B^2/n_B}} \quad (2.20)$$

Con grados de libertad estimados mediante la aproximación de Welch–Satterthwaite. El valor p bilateral es la probabilidad, bajo la hipótesis de medias iguales, de observar un estadístico al menos tan extremo como el obtenido. Un p pequeño indica evidencia contra esa hipótesis, pero no mide relevancia práctica; en las figuras se representa

$\log_{10} p$.

- **Cambio de distribución:** la divergencia de Kullback–Leibler entre dos distribuciones discretas p y q se define como:

$$D_{\text{KL}}(p \parallel q) = \sum_a p(a) \log \frac{p(a)}{q(a)} \quad (2.21)$$

La misma vale 0 cuando ambas coinciden, crece con la discrepancia y no es simétrica. En mPES se aplica de dos formas: entre la distribución empírica de acciones de un escenario y la de la condición de referencia del mismo modelo y en forma simetrizada $\frac{1}{2}[D_{\text{KL}}(p \parallel q) + D_{\text{KL}}(q \parallel p)]$, entre los histogramas de rendimiento de dos modelos. En ambos casos se suma una constante $\varepsilon = 10^{-9}$ y se renormaliza para evitar $\log 0$.

3. Estado de la Cuestión

La presente investigación se sitúa en la intersección de la epidemiología computacional [27], la neurociencia cognitiva de la confianza y el aprendizaje profundo secuencial. La literatura reciente muestra interés en sistemas de soporte para la toma de decisiones que, además de optimizar una métrica de rendimiento, informen sobre la *confianza* de sus decisiones.

3.1. Fundamentos neurocognitivos de la confianza algorítmica

El punto de partida conceptual de este trabajo es el conjunto de datos público ds004477 [28], publicado en OpenNeuro, y el entorno *Pandemic Experiment Scenario* (PES) [1] liberado por el BCI-NE Lab de la Universidad de Essex. Esta línea de investigación examinó las correlaciones neuronales de la confianza humana en decisiones bajo presión temporal. Boldt y Yeung [21] identificaron señales de EEG asociadas al monitoreo de errores y al ajuste de la certeza sobre una acción; Fleming [22] revisa la relación entre confianza, análisis secuencial y teoría de detección de señales en tareas de alta carga cognitiva.

La perspectiva de los equipos humano–máquina amplía este planteo al considerar que la toma de decisiones puede distribuirse entre personas y sistemas artificiales, de modo que

la confianza y la adaptación constituyen propiedades relevantes del equipo completo [29].

La contribución de mPES en este punto consiste en trasladar estas inquietudes al plano algorítmico: en lugar de una medida fisiológica, se utiliza la entropía de Shannon (Ecuación 2.15) de la distribución de acciones como indicador de incertidumbre del agente. Se trata de una analogía funcional; el trabajo no mide señales fisiológicas ni establece una equivalencia entre ambas magnitudes.

3.2. Aprendizaje por Refuerzo en la Gestión de Crisis

La literatura en RL ha evolucionado desde métodos tabulares como el Q-Learning [4] hacia aproximaciones capaces de manejar la alta dimensionalidad de los problemas del mundo real [3].

- **Deep Q-Networks (DQN).** La introducción de la repetición de experiencia y de la red objetivo por Mnih *et al.* [7] permitió estabilizar el aprendizaje con aproximadores no lineales. El operador máx del blanco de Q-Learning introduce un sesgo de sobreestimación que Double Q-Learning [5] mitiga al desacoplar la selección y la evaluación de la acción; Double DQN [10] traslada este desacople al caso profundo reutilizando la red objetivo como evaluador, y es la regla que adoptan los agentes DQN de mPES.
- **Estabilización del entrenamiento.** En los agentes profundos de mPES se emplean el optimizador Adam [9], recorte de la norma del gradiente y un esquema de exploración ε -greedy con fase inicial constante y decaimiento exponencial, cuyos parámetros se fijan por optimización bayesiana.
- **Actor–Critic (A2C).** Los métodos actor–crítico [15] reducen la varianza del gradiente de política mediante una línea de base aprendida (el crítico) y permiten trabajar directamente con una distribución de probabilidad sobre acciones, lo que facilita el cálculo de su entropía.

3.3. Modelado secuencial mediante arquitecturas Transformer

En el entorno estudiado, una asignación insuficiente en un paso se amplifica en los pasos siguientes por el factor $\beta > 1$ de la regla de transición. Una política que observa únicamente el estado instantáneo (R, t, S) puede representar esta dinámica solo de forma indirecta; los codificadores que reciben una ventana de estados pasados disponen de información adicional sobre la tendencia de la severidad.

La arquitectura Transformer [17] y su adaptación al RL como problema de modelado de secuencias en el *Decision Transformer* [19] proporcionan un mecanismo para explotar esa información: la auto-atención causal (Ecuación 2.11) pondera todas las posiciones previas de la ventana al construir la representación del paso actual. Parisotto *et al.* [18] mostraron que, con una disposición adecuada de la normalización y compuertas en los bloques, los Transformers igualan o superan a las LSTM en tareas de RL que requieren memoria.

3.4. Explicabilidad y cuantificación de incertidumbre

La adopción de sistemas de decisión automática en salud pública está condicionada por la posibilidad de interpretar sus salidas. Una medida de incertidumbre asociada a cada decisión permite señalar los casos en los que el modelo distribuye su preferencia entre varias acciones, lo que resulta útil para priorizar la supervisión humana. En esta tesis esa medida se obtiene de la entropía de la salida de los agentes; su relación con la incertidumbre humana se plantea como analogía y no como equivalencia.

3.5. Generalización bajo perturbaciones controladas

Diversos trabajos en RL profundo han señalado que las métricas obtenidas en las mismas condiciones del entrenamiento sobreestiman la calidad de las políticas aprendidas. Henderson *et al.* [30] mostraron que los algoritmos habituales son sensibles a la semilla aleatoria, a los hiperparámetros y a los detalles de implementación; Cobbe *et al.* [31] documentaron, sobre la suite ProcGen, que agentes con buen rendimiento en los niveles de entrenamiento fracasaban en niveles generados con la misma regla pero no vistos;

Packer *et al.* [32] distinguieron entre interpolación y extrapolación de los parámetros del entorno como formas de evaluar la generalización; y Kirk *et al.* [33] sistematizaron estas nociones en una revisión que recomienda evaluar sobre familias de variaciones controladas del entorno.

El diseño del benchmark de mPES (Sec. 4) sigue esa línea: dos suites de seis modelos evaluados sobre 22 escenarios cada una, cuatro familias de perturbación, contrastes de Welch reportados como $\log_{10} p$, columnas estructurales como comprobación del protocolo de agregación y degradación frente a la condición de referencia. En lugar de declarar un modelo ganador en la condición de referencia, el barrido completo muestra cómo cambia el rendimiento de cada arquitectura bajo perturbaciones controladas (Sec. 6.5).

3.6. Posición de este trabajo

Comparado con la literatura precedente, la contribución de esta tesis es:

- **Dos suites, un mismo esquema experimental.** Una comparación que abarca métodos tabulares (Q-Learning, Double Q-Learning), profundos densos (DQN), recurrentes (DQN con LSTM), actor-crítico (A2C), Transformer causal y seis variantes de ensamble, evaluados sobre el mismo entorno y $n = 64$ secuencias por escenario en 22 condiciones.
- **Indicador de confianza basado en entropía.** El uso de $1 - H_{\text{norm}}$ como indicador de confianza de cada agente y como factor de ponderación en la decisión conjunta de los ensambles, planteado por analogía con el estudio de la confianza humana en decisiones grupales.
- **Protocolo estadístico uniforme.** Cada diferencia reportada se acompaña de un tamaño de efecto (d de Cohen), un valor p de Welch y una divergencia de Kullback-Leibler que permite inspeccionar cambios en la distribución de acciones (Sec. 6.5).

4. Materiales y Métodos

Este capítulo describe el aparato experimental: el *workspace* mPES, la definición formal del entorno, los doce modelos algorítmicos comparados, el procedimiento de optimización

bayesiana de hiperparámetros y el protocolo de evaluación.

4.1. Repositorio y reproducibilidad

Todos los experimentos se ejecutan dentro del *workspace* `mPES` [34], derivado y extendido a partir del repositorio `PES` del grupo BCI-NE [1], del cual hereda la definición original del entorno *Pandemic Scenario* y la infraestructura tabular de Q-Learning. Es un proyecto Python organizado en tres grupos de paquetes: `tabular/` (métodos enumerativos), `ml/` (aprendizaje profundo) y `ens/` (ensambles de inferencia). Cada paquete es autocontenido: provee su propio entorno `gymnasium` [2], su rutina de entrenamiento, su módulo de optimización bayesiana cuando corresponde y su carpeta `outputs/` versionada por fecha. Las dependencias principales son TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

4.2. Especificación del entorno *Pandemic Scenario*

4.2.1. Definición formal

El entorno modela una pandemia simplificada como un MDP finito en el que el agente decide, en cada *trial*, cuántas unidades de un recurso escaso destinar a la ciudad que se incorpora en ese paso. El espacio de estados observado por el agente es:

$$\mathcal{S} = \{(R, t, S) : R \in \{0, \dots, 30\}, t \in \{0, \dots, T_{\text{máx}}\}, S \in \{0, \dots, S_{\text{máx}}\}\} \quad (4.1)$$

Donde R son los recursos restantes, t el índice de *trial* dentro de la secuencia ($t = 0$ en el estado inicial, $T_{\text{máx}} = 10$) y S la severidad inicial de la ciudad actual ($S_{\text{máx}} = 9$); el número de estados es $|\mathcal{S}| = 31 \times 11 \times 10 = 3410$. La formulación como sistema dinámico de decisiones se relaciona con los modelos en los que las decisiones actuales modifican el estado y las opciones disponibles en etapas posteriores [35]. En este trabajo esa estructura se instancia en un modelo epidemiológico simplificado utilizado como banco de pruebas, no como una reproducción de una pandemia real. El presupuesto total de cada secuencia es de 39 recursos; como 9 se asignan de forma predeterminada, el agente administra $R \in \{0, \dots, 30\}$. El espacio de acciones es $\mathcal{A} = \{0, 1, \dots, 10\}$; las asignaciones

que exceden R se recortan a los recursos disponibles.

4.2.2. Dinámica y recompensa

En cada paso, la severidad de toda ciudad i ya presente en la secuencia se actualiza mediante la recurrencia lineal con saturación en cero:

$$S_{i,t+1} = \max(0, \beta S_{i,t} - \alpha a_i), \quad \alpha = 0,4 \quad \beta = 1 + \alpha = 1,4 \quad (4.2)$$

Donde a_i es la asignación recibida por la ciudad i en el paso en que se incorporó, α la eficacia marginal de cada unidad de recurso y $\beta > 1$ el factor de crecimiento por paso. **Como la asignación de una ciudad se fija una sola vez y su severidad se actualiza en todos los pasos posteriores, una asignación insuficiente se amplifica geométricamente.** La recompensa instantánea es la suma negativa de las severidades de las ciudades activas tras la actualización:

$$r_t = - \sum_{i \leq t} S_{i,t+1} \quad (4.3)$$

De este modo maximizar el retorno equivale a minimizar la severidad acumulada de la secuencia.

4.2.3. Estructura del benchmark

Cada experimento del *workspace* mPES se organiza jerárquicamente en **8 bloques** de **8 secuencias** de **3–10 trials** cada una, totalizando ≈ 360 trials por ejecución. La evaluación final se realiza sobre $n = 64$ secuencias fijas (8×8 , semilla = 42), garantizando que los modelos de cada suite enfrenten el *mismo* conjunto de secuencias en forma pseudoaleatoria. El catálogo incluye **22 escenarios**: una condición de referencia y 21 condiciones de perturbación, agrupadas en cuatro familias — *severidad* (9), *longitud* (5), *conjunta* (4) y *estructural* (3) — de modo que el barrido completo del benchmark (`general/scripts/benchmark.py`) abarca dos suites de $6 \times 22 = 132$ celdas, es decir, 264 celdas en total; `pes_base` se conserva como referencia del entrenamiento histórico y no forma parte del ranking comparativo. Un agente aleatorio (Sec. 4.5) fija además el piso

de desempeño del banco de pruebas.

4.3. Modelos algorítmicos

La Tabla 4.1 resume los seis modelos individuales y las seis variantes de ensamble evaluadas; `pes_base` se muestra como referencia adicional. El repositorio `mPES` organiza estas variantes en dos familias: *tabular* (métodos enumerativos con representación exacta del MDP) y *profunda* (redes neuronales entrenadas por descenso de gradiente). Los modelos individuales comparten el entorno descrito en la Ecuación (4.2); los modelos optimizables emplean Optuna/TPE. Los ensambles se ejecutan en inferencia y cargan artefactos preentrenados. Todas las variantes se comparan mediante el mismo protocolo de escenarios y secuencias.

Cuadro 4.1: Paquetes y familias algorítmicas incluidos en `mPES`, con la característica distintiva de cada implementación. El benchmark evalúa doce modelos comparativos en dos suites de seis; `pes_base` se conserva como referencia del entrenamiento histórico.

Paquete	Familia	Característica distintiva
<code>pes_base</code>	Tabular	Q-Learning de referencia.
<code>pes_q1</code>	Tabular	Q-Learning + Optuna/TPE.
<code>pes_dq1</code>	Tabular	Double Q-Learning + PBRs + ε <i>warm-up</i> .
<code>pes_dqn</code>	Profundo	Double DQN con <i>replay</i> y red objetivo.
<code>pes_rdqn</code>	Profundo	Double DQN recurrente con codificador LSTM.
<code>pes_a2c</code>	Profundo	Actor-Crítico (A2C).
<code>pes_trf</code>	Profundo	Codificador Transformer causal sobre historia.
<code>pes_ens</code>	Ensamble	Votación ponderada de modelos profundos.
<code>pes_ens_sprb</code>	Ensamble	Votación probabilística ponderada.
<code>pes_ens_accq</code>	Ensamble	Votación de acciones con desempate por Q.
<code>pes_ens_consensus</code>	Ensamble	Consenso con acuerdo y desacuerdo.
<code>pes_ens_consensus_prior</code>	Ensamble	Consenso con prior de severidad.
<code>pes_ens_trf_guard</code>	Ensamble	Transformer con compuerta y respaldo.

4.3.1. Familia tabular

Los tres agentes tabulares representan explícitamente la función $Q(s, a)$ mediante una tabla. El estado es $s = (R, t, S) \in \{0, \dots, 30\} \times \{0, \dots, 10\} \times \{0, \dots, 9\}$, por lo que la tabla tiene $31 \times 11 \times 10 \times 11 = 37\,510$ valores. Las acciones son $a \in \{0, \dots, 10\}$ y las que exceden los recursos disponibles se enmascaran. En cada paso se selecciona una acción mediante una política ε -greedy, se observa (s', r) y se actualiza la entrada visitada. Para Q-Learning estándar, la actualización es

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.4)$$

salvo en un estado terminal, donde se utiliza $Q(s, a) \leftarrow r$. Las tablas se inicializan con valores de una distribución uniforme $\mathcal{U}(-1, 1)$ y la evaluación utiliza la acción válida con mayor valor. Esta especificación, junto con la dinámica del entorno definida en la Sección 4, determina el procedimiento reproducible de los modelos tabulares.

4.3.1.1 pes_base

Es Q-Learning estándar con tasa de aprendizaje $\alpha = 0,2$, factor de descuento $\gamma = 0,9$, exploración inicial $\varepsilon_0 = 0,8$ y mínimo $\varepsilon_{\min} = 0$. La exploración decrece linealmente durante los episodios de entrenamiento hasta alcanzar ese mínimo. El número predeterminado de episodios es 1 000 000; la semilla experimental es 42. Este agente no emplea Double Q-Learning, warm-up ni PBRS y sirve de línea de base inferior.

4.3.1.2 pes_ql

Mantiene la misma tabla y actualización, pero añade un bucle externo de Optuna/TPE. Cada configuración se entrena y se evalúa sobre el mismo conjunto de validación; la función objetivo es el desempeño normalizado medio. Optuna muestrea $\alpha \in [0,05, 0,40]$ en escala logarítmica, $\gamma \in [0,85, 0,999]$, $\varepsilon_0 \in [0,50, 1,00]$, $\varepsilon_{\min} \in [0,01, 0,15]$ y entre 500 000 y 1 200 000 episodios, en incrementos de 50 000. La política de exploración utiliza el mismo decaimiento lineal que el modelo base; el resto del algoritmo permanece sin modificaciones. El mejor de los 100 ensayos (índice 63, semilla de entrenamiento 106) fijó $\alpha = 0,2862$, $\gamma = 0,8587$, $\varepsilon_0 = 0,6813$, $\varepsilon_{\min} = 0,0437$ y 550 000 episodios, con un desempeño normalizado medio de 0,887 sobre las 64 secuencias de validación; con esa configuración se reentrenó el agente final.

4.3.1.3 pes_dql

Implementa Double Q-Learning [5], una fase de calentamiento de ε y PBRS con $\Phi(s) = -S$ [6], orientado a acelerar la convergencia ante una señal de recompensa cuya magnitud se concentra en los últimos pasos de la secuencia. En concreto, mantiene dos tablas independientes Q_A y Q_B , inicializadas en $\mathcal{U}(-1, 1)$. En cada paso se actualiza una sola

tabla, elegida con probabilidad $1/2$; si se elige Q_A , la acción siguiente se selecciona con Q_A y se evalúa con Q_B , y viceversa. La política final es $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$. El decaimiento de exploración es exponencial después de un warm-up: permanece en ε_0 durante wN episodios y alcanza $\varepsilon_{\min}$ en qN , con:

$$\lambda = \left(\frac{\varepsilon_{\min}}{\varepsilon_0} \right)^{1/((q-w)N)} \quad \varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \lambda^{t-wN}) \quad \text{para } t \geq wN \quad (4.5)$$

Donde t es el índice de episodio, N el número total de episodios, w la fracción de calentamiento y q la fracción en la que se alcanza $\varepsilon_{\min}$. Los hiperparámetros se fijaron con un estudio Optuna/TPE de 100 ensayos (14 completados y 86 podados); el mejor (índice 3, semilla de entrenamiento 46) obtuvo un desempeño normalizado medio de 0,896 con $\alpha = 0,1132$, $\gamma = 0,9777$, $\varepsilon_0 = 0,5998$, $\varepsilon_{\min} = 0,0327$, $N = 360\,000$, $w = 0,0260$ (9 373 episodios de calentamiento), $q = 0,6430$ ($\varepsilon_{\min}$ alcanzado en el episodio 231 486) y coeficiente PBRS $\kappa = 0,000392$. Con $\Phi(s) = -S$, la recompensa utilizada para actualizar la tabla completando las definiciones del agente son:

$$\tilde{r} = r + \kappa [\gamma \Phi(s') - \Phi(s)] \quad (4.6)$$

4.3.2. Familia profunda

4.3.2.1 Representación del estado.

Los cuatro agentes profundos reciben el estado en forma normalizada: cada componente entera de (R, t, S) se divide por su cota superior en el entorno, de modo que la entrada de la red es $s = [R/30, t/10, S/9] \in [0, 1]^3$. Se trata de un escalado min-max fijo por las cotas del MDP, no de una estandarización estadística, y es exclusivo de esta familia: los agentes tabulares indexan la tabla directamente con los enteros. El escalado responde a un requisito numérico del entrenamiento por descenso de gradiente. Sin él, la componente R (rango 0–30) dominaría a t y S (rangos 0–10 y 0–9) en las activaciones de la primera capa, y los rangos dispares interactúan mal con la inicialización de los pesos y con la tasa de aprendizaje única de Adam; llevar las tres entradas a $[0, 1]$ equilibra su contribución inicial y evita ajustar la escala por separado para cada una. La transformación se aplica en todas las fases del ciclo de vida del modelo —entrenamiento (tanto al elegir la acción como al

almacenar las transiciones en el *buffer*), optimización bayesiana, evaluación determinista, barrido de escenarios e inferencia de los ensambles, que reproducen la misma función al cargar los modelos guardados— y, por ser una función determinista de las cotas del entorno, no introduce fuga de información entre entrenamiento y evaluación. La factibilidad de las acciones se recupera invirtiéndola ($R = 30 s_1$) donde hace falta enmascarar.

4.3.2.2 Efecto sobre la comparación entre familias.

La normalización es interna al agente y no altera ninguna de las magnitudes sobre las que se construye el análisis. El entorno, la dinámica de la Ecuación (4.2), la recompensa, el espacio de acciones $\mathcal{A} = \{0, \dots, 10\}$ y las 64 secuencias de evaluación son idénticos para todas las familias; la red recibe s escalado pero emite una acción entera, que el entorno procesa exactamente igual que la de una tabla Q . El desempeño normalizado $\bar{r}$ (Ecuación (4.15)) se calcula a partir de las severidades finales que produce el entorno, con las mismas referencias S_{peor} y S_{mejor} por secuencia, con independencia de cómo el agente represente su entrada. En consecuencia, la media $\bar{r}$ por escenario, los 64 valores por secuencia sobre los que operan la prueba de Welch, la d de Cohen y la KL de rendimiento, y las distribuciones empíricas de acciones sobre las que se calcula la KL de acciones son directamente comparables entre agentes tabulares, profundos y ensambles: la normalización cambia la parametrización de la función de valor, no la escala ni el significado de lo que se mide.

4.3.2.3 Configuración compartida.

Los cuatro agentes utilizan el optimizador Adam, inicialización Glorot uniforme y recorte de la norma global del gradiente. La semilla de entrenamiento de cada modelo es $42 + i + 1$, con i el índice del mejor ensayo de su estudio Optuna, y se aplica a la inicialización de pesos, al muestreo del *buffer* y a la exploración; la generación de secuencias de evaluación usa siempre la semilla 42. Los agentes DQN, RDQN y Transformer comparten la misma regla de actualización —el blanco Double DQN de la Ecuación 2.6, con la acción de *bootstrap* seleccionada por la red en línea entre las acciones factibles en s' y evaluada por la red objetivo, y pérdida de Huber sobre el error temporal—, el mismo esquema de exploración ε -greedy con fase constante durante una fracción w de los episodios y decaimiento exponencial hasta $\varepsilon_{\text{mín}}$ en la fracción q (Ecuación (4.5)), el mismo umbral

de inicio del aprendizaje (una fracción f de la capacidad del *buffer*, con mínimo de un lote) y PBRs con $\Phi(s) = -S$ cuando el coeficiente κ es positivo. Los valores concretos difieren entre los tres y se listan en cada párrafo; en todos los casos corresponden al mejor ensayo de la optimización bayesiana del paquete, con el que se reentrenó el modelo final utilizado en el benchmark. A2C se entrena *on-policy* y sus hiperparámetros se detallan en su párrafo.

4.3.2.4 pes_dqn

Red densa con repetición de experiencia [8] y red objetivo [7], entrenada con el blanco Double DQN [10] de la Ecuación 2.6. La arquitectura es $\text{Input}(3) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(11)$ (5 131 parámetros), y la salida contiene un valor Q para cada una de las 11 acciones. El estudio Optuna comprendió 47 ensayos; el mejor (índice 41, semilla 84, desempeño medio 0,894) fijó tasa de aprendizaje 0,001508, $\gamma = 0,9634$, $\varepsilon_0 = 0,9627$, $\varepsilon_{\min} = 0,0691$, $w = 0,2779$, $q = 0,6290$, lotes de 128 transiciones, *buffer* de 20 000, sincronización de la red objetivo cada $C = 1\,000$ pasos, recorte de gradiente en 3,953, $\kappa = 0,0226$ e inicio del aprendizaje en $f = 0,1615$ (3 230 transiciones). El modelo final se entrenó durante 40 000 episodios.

4.3.2.5 pes_rdqn

Variante recurrente que incorpora una capa LSTM [12] entre la entrada y el cabezal de valores, siguiendo el esquema DRQN [13]: la red recibe una ventana de los últimos $W = 6$ estados normalizados $(s_{t-5}, \dots, s_t)$, completada con ceros al inicio de cada secuencia, recorre la ventana con la LSTM de 64 unidades partiendo de un estado oculto nulo y produce $Q(h_t, \cdot)$ a partir del último estado oculto h_t (Ec. 2.7) mediante dos capas densas de 96 unidades con ReLU y una capa de salida de 11 valores (34 027 parámetros). El estado oculto no persiste entre pasos de decisión: cada ventana se procesa de manera independiente, tanto en la interacción con el entorno como en las transiciones muestreadas del *buffer*, que almacenan la ventana completa. Esto permite incorporar la evolución reciente de la severidad, que no está contenida en el estado instantáneo s_t . El estudio Optuna se lanzó con 20 ensayos; el mejor (índice 14, semilla 57, desempeño medio 0,926) fijó tasa de aprendizaje 0,002415, $\gamma = 0,9719$, $\varepsilon_0 = 0,8883$, $\varepsilon_{\min} = 0,0887$, $w = 0,1237$, $q = 0,5990$, lotes de 64 transiciones, *buffer* de 60 000, $C = 2\,000$ pasos, recorte de gradiente

en 3,475, $\kappa = 0,00263$ y $f = 0,2021$ (12 128 transiciones). La longitud de ventana $W = 6$ y la anchura de la LSTM (64) se mantienen fijas en la configuración del paquete y no forman parte del espacio optimizado. El modelo final se entrenó durante 30 000 episodios.

4.3.2.6 pes_a2c

Actor y crítico como redes independientes: $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(11, \text{softmax})$ para el actor y $\text{Input}(3) \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(1)$ para el crítico. La ventaja se estima con GAE (Ecuación 2.9) y el gradiente de política (Ecuación 2.8) se calcula sobre la distribución restringida a las acciones factibles [15]. El actor produce directamente una distribución de probabilidad sobre las 11 acciones. El estudio Optuna comprendió 100 ensayos; el mejor (índice 89, semilla 132, desempeño medio 0,887) fijó tasas de aprendizaje iniciales 0,000648 (actor) y 0,004299 (crítico), ambas con decaimiento coseno a lo largo del entrenamiento hasta el 23,75 % del valor inicial, $\gamma = 0,8545$, coeficiente de entropía $\beta_H = 0,00528$, $\lambda = 0,9135$ en GAE, recorte de la norma del gradiente en 1,200 para ambas redes, PBRs con $\kappa = 0,1527$, una penalización de gasto $r \leftarrow r - 0,01039 a$ y un sesgo inicial de $-1,389$ en el logit de la acción $a = 10$. Las ventajas GAE se tipifican por lote antes de la actualización del actor. El modelo final se entrenó durante 125 000 episodios.

4.3.2.7 pes_trf

Codificador Transformer causal de $L = 2$ bloques, $h = 4$ cabezas de atención con dimensión de clave 16, anchura de representación $d_{\text{model}} = 32$, capa *feed-forward* de 64 unidades y sin *dropout*. La entrada es la misma ventana de $W = 6$ estados normalizados que en **pes_rdn**, proyectada a d_{model} y sumada a un vector posicional fijo por posición, muestreado con inicialización Glorot uniforme a partir de la semilla y no actualizado durante el entrenamiento (sus 192 valores no forman parte de los parámetros entrenables del modelo); la máscara causal (Ecuación 2.11) impide que una posición atienda a estados posteriores. La representación de la última posición alimenta una capa densa de 32 unidades con ReLU y una capa de salida con los 11 valores Q (27 019 parámetros entrenables). Cada bloque aplica atención de producto escalar escalado (Ecuación 2.10) [17] con conexión residual y normalización de capa. El estudio Optuna comprendió 10 ensayos; el mejor (índice 2, semilla 45, desempeño medio 0,925) fijó tasa de aprendizaje 0,000215, $\gamma = 0,9234$,

$\varepsilon_0 = 0,8651$, $\varepsilon_{\min} = 0,0838$, $w = 0,2428$, $q = 0,5333$, lotes de 128 transiciones, *buffer* de 30 000, $C = 500$ pasos, recorte de gradiente en 4,170, PBRS desactivado ($\kappa = 0$) y $f = 0,1217$ (3 650 transiciones). La arquitectura del codificador (W , d_{model} , h , dimensión de clave, anchura *feed-forward*, L y *dropout*) se mantiene fija en la configuración del paquete al reentrenar el modelo final, que se entrenó durante 30 000 episodios. Sobre los valores Q factibles se calcula la medida de confianza basada en entropía descrita en la Sec. 2.

4.3.3. Ensamblajes

Los ensambles no se entrenan: cargan los modelos guardados de la familia profunda y combinan sus salidas durante la inferencia. Los seis paquetes (`pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`) reutilizan los artefactos finales de `pes_dqn`, `pes_rdqn` y `pes_trf`; las cinco variantes optimizables incorporan además el actor de `pes_a2c` como cuarto miembro, cuya salida *softmax* se usa directamente como distribución sobre acciones, mientras que `pes_ens` lo mantiene deshabilitado.

4.3.3.1 `pes_ens`.

Votación ponderada en inferencia sobre los modelos guardados de `pes_dqn`, `pes_rdqn` y `pes_trf`. Cada miembro k convierte sus valores Q en una distribución mediante una *softmax* con temperatura τ , $p_k(a | s) = \text{softmax}(Q_k(s, \cdot)/\tau)_a$, y el ensamble combina

$$\pi_{\text{ens}}(a | s) \propto \sum_{k=1}^K w_k p_k(a | s), \quad \sum_k w_k = 1, \quad (4.7)$$

con $K = 3$, $\tau = 15$ y pesos base 0,18, 0,90 y 5,0 (renormalizados) para DQN, RDQN y Transformer respectivamente. La implementación añade heurísticas fijadas en la configuración: las acciones que exceden los recursos disponibles se enmascaran y la distribución se renormaliza; el peso de cada miembro se multiplica por $0,1 + c_k$, con $c_k = 1 - H_{\text{norm}}(p_k)$ su propia confianza; la distribución resultante se mezcla, con peso 0,17, con una gaussiana centrada en la severidad actual ($\sigma = 3$); y una cota de seguridad fuerza $a = \lfloor S/2 \rfloor$ cuando $S \geq 6$ y la acción elegida es menor que esa cota. El paquete no introduce parámetros entrenables, en línea con la formulación de *deep*

ensembles de Lakshminarayanan *et al.* [20]. Las cinco variantes restantes (`pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`, `pes_ens_consensus_prior` y `pes_ens_trf_guard`) modifican la regla de combinación (votación por acción con desempate por valores Q normalizados, consenso ponderado por confianza con términos de acuerdo y desacuerdo, o prioridad del Transformer con respaldo condicional) y exponen sus parámetros a un módulo `optimize_ens.py`.

4.3.3.2 Variantes de combinación.

Las cinco variantes adicionales comparten la misma notación. Para cada miembro k se obtiene, sobre las acciones factibles, la distribución $p_k(a | s)$ (Ecuación (2.12); para el actor de A2C se toma su salida *softmax* renormalizada), la acción preferida $a_k = \arg \max_a p_k(a | s)$, la confianza $c_k = 1 - H_{\text{norm}}(p_k)$ y el peso efectivo $\tilde{w}_k = w_k c_k^\rho$, con ρ el exponente de confianza. Además se define la versión tipificada de los valores de cada miembro, $\hat{Q}_k(a) = (Q_k(a) - \bar{Q}_k) / \text{sd}(Q_k)$, utilizada para comparar salidas de escalas distintas. Salvo en `pes_ens_sprb`, la *softmax* sobre valores Q se aplica con $\tau = 1$. Los pesos w_k , el exponente ρ y los parámetros propios de cada regla se fijaron con Optuna sobre las 64 secuencias de validación; el peso del actor de A2C sólo se optimiza en `pes_ens_consensus_prior` y en las otras cuatro variantes conserva su valor por defecto $w_{\text{a2c}} = 0,10$. Las cinco reglas se diferencian en cómo agregan estas cantidades.

- `pes_ens_sprb` (voto suave probabilístico) promedia las distribuciones ponderadas por el peso efectivo,

$$p_{\text{sprb}}(a | s) = \frac{\sum_k \tilde{w}_k p_k(a | s)}{\sum_k \tilde{w}_k}, \quad a_{\text{sprb}} = \arg \max_a p_{\text{sprb}}(a | s). \quad (4.8)$$

Parámetros optimizados: $\tau = 0,1044$, $\rho = 0,4677$, $w_{\text{dq}} = 2,852$, $w_{\text{rdq}} = 1,628$, $w_{\text{trf}} = 1,134$.

- `pes_ens_accq` (voto por acción con desempate por Q) acumula un voto ponderado por la acción preferida de cada miembro y resuelve empates con la suma de valores tipificados:

$$V(a) = \sum_k \tilde{w}_k \mathbf{1}[a_k = a], \quad T(a) = \sum_k \hat{Q}_k(a), \quad (4.9)$$

$$a_{\text{accq}} = \arg \max_{a \in \mathcal{V}} T(a), \quad \mathcal{V} = \{a : V(a) = \max_{a'} V(a')\}. \quad (4.10)$$

Parámetros optimizados: $\rho = 1,005$, $w_{\text{dqn}} = 2,374$, $w_{\text{rdqn}} = 0,722$, $w_{\text{trf}} = 1,008$.

- **pes_ens_consensus** (consenso por confianza) parte del mismo voto ponderado y le suma un término de acuerdo, uno de desacuerdo y el consenso de valores tipificados:

$$S(a) = \sum_k \tilde{w}_k \mathbf{1}[a_k = a] + \beta_a \sum_{k: a_k = a} c_k - \beta_d \sum_{k: a_k \neq a} c_k + \sum_k \hat{Q}_k(a) c_k, \quad a_{\text{cons}} = \arg \max_a S(a), \quad (4.11)$$

con bono de acuerdo β_a y penalización de desacuerdo β_d . La regla favorece las acciones apoyadas por miembros confiados y penaliza el desacuerdo. Parámetros optimizados: $\beta_a = 0,1649$, $\beta_d = 0,3647$, $\rho = 0,4026$, $w_{\text{dqn}} = 2,995$, $w_{\text{rdqn}} = 0,320$, $w_{\text{trf}} = 1,695$.

- **pes_ens_consensus_prior** (consenso con prior de severidad) usa una acumulación suave, $\tilde{S}(a) = \sum_k \tilde{w}_k p_k(a | s)$, más los mismos términos de acuerdo y desacuerdo, atenua la acción nula ($\tilde{S}(0) \leftarrow 0,3 \tilde{S}(0)$ cuando hay recursos) y la normaliza. La distribución resultante se mezcla con un prior gaussiano centrado en la severidad actual S ,

$$\pi_{\text{prior}}(a | s) \propto \exp \left[-\frac{(a - S)^2}{2\sigma_S^2} \right], \quad S(a) = (1 - w_\pi) \tilde{S}(a) + w_\pi \pi_{\text{prior}}(a | s), \quad (4.12)$$

con peso del prior w_π y anchura σ_S . Una cota de seguridad fuerza $a = \lfloor S/2 \rfloor$ cuando $S \geq 6$ y la acción elegida es menor que esa cota. Parámetros optimizados: $w_\pi = 0,2051$, $\sigma_S = 0,9177$, $\beta_a = 1,810$, $\beta_d = 0,3462$, $\rho = 0,4180$, $w_{\text{dqn}} = 2,778$, $w_{\text{a2c}} = 0,1044$, $w_{\text{rdqn}} = 2,465$, $w_{\text{trf}} = 2,840$.

- **pes_ens_trf_guard** (compuerta del Transformer) calcula una compuerta logística sobre la confianza del Transformer,

$$g = \frac{1}{1 + \exp[-\kappa (c_{\text{trf}} - \tau_g)]}, \quad f(a) = \sum_k \tilde{w}_k p_k(a | s), \quad (4.13)$$

y selecciona la acción del Transformer cuando $g \geq 0,5$ y la del voto de respaldo f en caso contrario: $a_{\text{guard}} = a_{\text{trf}}$ si $g \geq 0,5$, y $a_{\text{guard}} = \arg \max_a f(a)$ si $g < 0,5$. El umbral τ_g y la pendiente κ son hiperparámetros del paquete. Parámetros optimizados:

$$\tau_g = 0,2248, \kappa = 3,964, \rho = 1,896, w_{\text{dqn}} = 1,124, w_{\text{rdqn}} = 2,852, w_{\text{trf}} = 2,196.$$

Estas reglas no se entrenan conjuntamente con los miembros. Por tanto, las diferencias entre variantes miden el efecto de la regla de combinación y de sus hiperparámetros sobre artefactos ya entrenados, no una comparación entre arquitecturas entrenadas de extremo a extremo.

4.4. Procedimiento de optimización bayesiana

Cada paquete optimizable expone un módulo `optimize_*.py` que ejecuta un estudio Optuna con estimador estructurado en árbol de Parzen (TPE) [23, 24]. Se trata específicamente de una optimización de **hiperparámetros**: decisiones de configuración fijadas antes de entrenar, como la tasa de aprendizaje, el factor de descuento, la exploración, el tamaño del lote, la longitud de contexto o la arquitectura de la red. Estos no deben confundirse con los parámetros aprendidos: los valores de la tabla Q y los pesos de una red se ajustan dentro de cada prueba y no son seleccionados directamente por Optuna. El paquete `pes_base` constituye la línea de base y no participa en esta búsqueda. Los fundamentos del método se introducen en la Sec. 2; aquí sólo se describe su instanciación en el banco de pruebas. Sea θ una configuración de hiperparámetros; para cada configuración se entrena desde cero el agente correspondiente y se evalúa su política resultante. La función objetivo $f(\theta)$ es el desempeño normalizado medio $\bar{r}$ (Ecuación (4.15)) sobre las $n = 64$ secuencias fijas, y se busca $\theta^* = \arg \max_{\theta} f(\theta)$. Los estudios incluyen poda temprana mediante `MedianPruner`, que interrumpe los ensayos cuya recompensa intermedia queda por debajo de la mediana de los ensayos previos, y conservan el almacenamiento SQLite (`optuna_storage.db`) en `outputs/`, lo que permite reanudar el estudio. El agente se reentrena desde cero con θ^* .

La comparación entre métodos y su comportamiento bajo perturbaciones se presenta en el capítulo de Resultados.

4.5. Referencia de comportamiento aleatorio

Antes de comparar arquitecturas conviene fijar el piso de desempeño. La Figura 4.1 muestra el comportamiento de un agente que elige acciones con distribución uniforme

sobre $\mathcal{A}$, evaluado sobre las mismas 64 secuencias.

La magnitud en escala cruda se denomina *severidad final acumulada*. Para una secuencia de T *trials*, si S_i^{final} es la severidad de la ciudad i al terminar la secuencia, se define

$$S_{\text{cruda}} = \sum_{i=1}^T S_i^{\text{final}}, \quad (4.14)$$

de modo que un valor menor representa un mejor resultado. En la Figura 4.1 la curva representa S_{cruda} del agente aleatorio, no una métrica normalizada.

Para comparar secuencias con distinta severidad inicial y distinta longitud se utiliza el *desempeño normalizado de severidad final*. Para cada secuencia se calculan dos referencias: S_{peor} , la severidad acumulada con asignación nula en todos los *trials*, y S_{mejor} , la mínima severidad acumulada alcanzable con asignaciones enteras en $\{0, \dots, 10\}$ que respeten el presupuesto de 30 recursos, obtenida por programación dinámica sobre las ciudades de la secuencia. Si S_{agente} es la severidad cruda del agente, la métrica es

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}}, \quad (4.15)$$

de modo que $\bar{r} = 0$ corresponde a no asignar recursos y $\bar{r} = 1$ a la mejor asignación factible. El agente aleatorio se ubica entre ambos extremos y sirve de referencia independiente de todo aprendizaje.

4.6. Variables internas del agente Transformer

La Figura 4.2 expone variables registradas por el script de entrenamiento de `pes_trf` al evaluar el modelo recién entrenado sobre las 64 secuencias: la confianza por decisión derivada de la entropía de los valores Q (unos 360 *trials*), su reescalado lineal al intervalo $[0, 1]$, el desempeño normalizado por secuencia y su media acumulada. Estas trazas se retoman en el Cap. 6 al discutir la relación entre la entropía y la noción de confianza estudiada en decisión humana [21, 22].

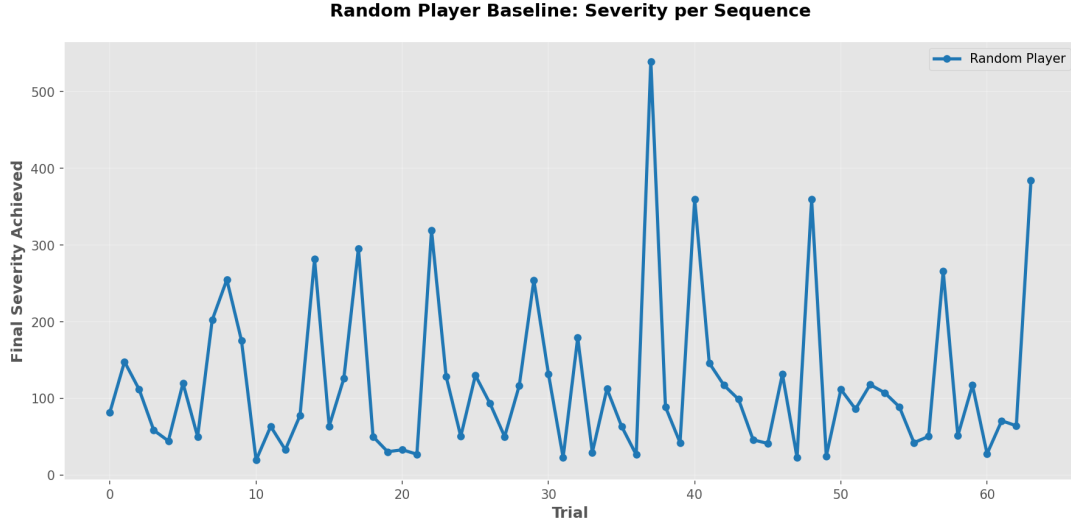


Figura 4.1: Severidad final acumulada S_{cruda} del decisor aleatorio en cada una de las 64 secuencias de validación. La escala vertical conserva las unidades originales de severidad y permite identificar la dispersión del caso sin asignación informada de recursos.

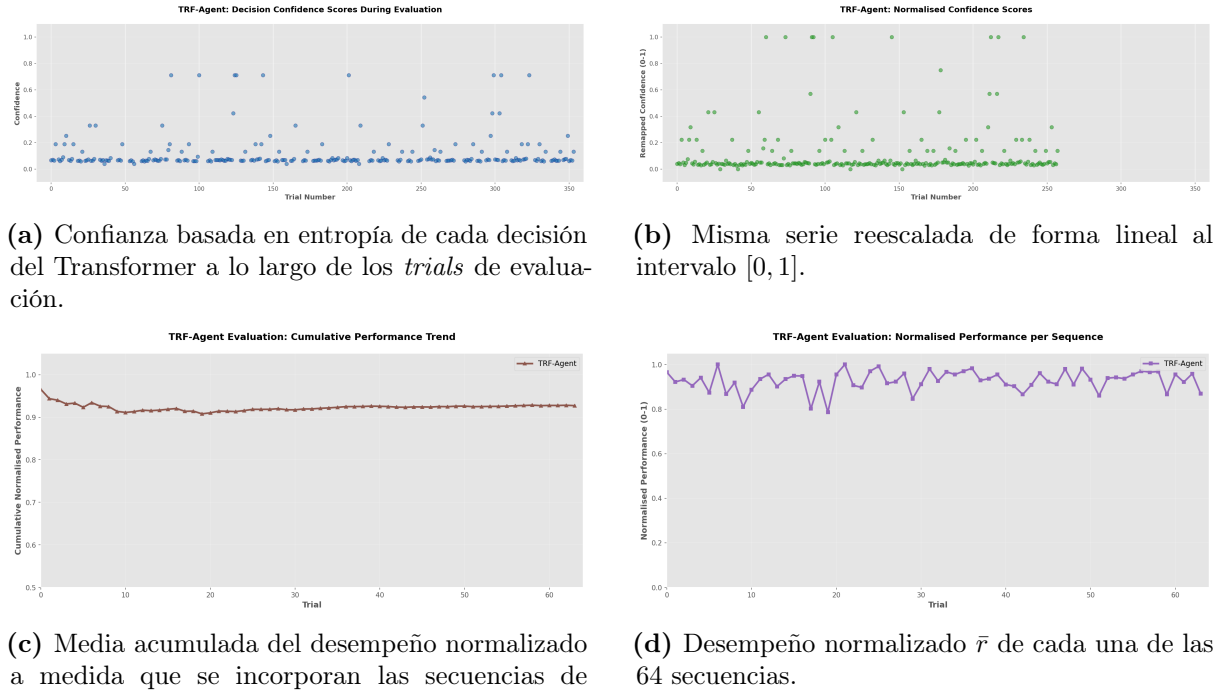


Figura 4.2: Variables registradas al evaluar el agente `pes_trf` tras su entrenamiento. Los paneles superiores muestran la confianza basada en entropía por decisión (cruda y reescalada a $[0, 1]$); los paneles inferiores, la media acumulada y el desempeño normalizado por secuencia.

4.7. Catálogo de escenarios de perturbación

El banco de pruebas `general/` (véase `general/README.md`) define un **catálogo de 22 escenarios** que evalúan cada modelo entrenado a lo largo de cuatro ejes de variación con respecto a la distribución empírica de entrenamiento (etiquetada `sev_base`). El armazón de benchmark (`general/scripts/benchmark.py`) ejecuta el producto cartesiano de seis

modelos por 22 escenarios en cada suite, es decir, 132 celdas por suite y 264 en total, evaluando $n = 64$ secuencias por celda con semilla fija 42 — lo que garantiza que los modelos de cada suite enfrenten exactamente las mismas secuencias dentro de cada escenario. La evaluación es de pura *inferencia*: ningún paquete se reentrena durante el barrido.

Las cuatro familias responden a preguntas distintas:

- **Severidad (9 escenarios)**. Sustituyen la distribución empírica de severidades iniciales por familias paramétricas (uniforme, gaussianas truncadas de media baja, media y alta, Weibull con cola superior pesada, Beta sesgadas y mezcla bimodal) y un caso fuera del soporte de entrenamiento (`sev_extrapolate_high`, $U\{10, \dots, 12\}$, cuando el entrenamiento cubre $\{0, \dots, 9\}$). Modifican la distribución de la señal de entrada manteniendo las longitudes de secuencia.
- **Longitud (5 escenarios)**. Modifican la distribución de longitudes de secuencia (todas de 3, todas de 10, geométrica, Poisson y un caso fuera del soporte con $U\{11, \dots, 20\}$, cuando el entrenamiento cubre $[3, 10]$). Evalúan la capacidad de sostener decisiones consistentes en horizontes más largos que los vistos en entrenamiento.
- **Conjunta (4 escenarios)**. Combinan severidad y longitud (alta–larga, baja–corta, uniforme–geométrica y un escenario con ambos factores fuera del soporte). Permiten observar interacciones entre ambos ejes que no son visibles al variarlos por separado.
- **Estructural (3 escenarios)**. Reconfiguran el número de bloques y de secuencias por bloque, incluyendo una variante con $n = 128$ secuencias. Comprueban que la agregación de resultados no depende de la partición en bloques.

La Tabla 4.2 enumera el catálogo completo. El escenario `sev_base` actúa como condición de referencia: conserva la ley de generación utilizada en el entrenamiento. Los restantes escenarios modifican una o más propiedades de esa ley. Frente a la referencia se calculan, para cada modelo y escenario, la *degradación* $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$ y la divergencia KL (Ecuación (2.21)) entre la distribución empírica de acciones de la celda y la de la referencia. La significancia del cambio se evalúa con la prueba t de Welch bilateral (Ecuación (2.20)) sobre los 64 valores por secuencia y el tamaño del efecto con la d

de Cohen (Ecuación (2.19)).

Cuadro 4.2: Catálogo de 22 escenarios del banco de pruebas **general/**. La fila **sev_base** actúa como condición de referencia. Los escenarios con etiqueta **extrapolate** caen fuera del soporte de entrenamiento (severidad en $\{10, 11, 12\}$ o longitud en $\{11, \dots, 20\}$).

Familia	Identificador	Descripción
baseline	sev_base	Distribución empírica de entrenamiento (referencia).
severidad	sev_uniform	$U\{0, \dots, 9\}$, sin estructura modal.
severidad	sev_gauss_low	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	sev_gauss_mid	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	sev_gauss_high	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	sev_weibull	Weibull($k = 1, 5$), cola superior pesada.
severidad	sev_beta_lowskew	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	sev_beta_highskew	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	sev_bimodal	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	sev_extrapolate_high	$U\{10, 11, 12\}$, fuera del soporte.
longitud	len_all_short	Todas las secuencias con longitud 3.
longitud	len_all_long	Todas las secuencias con longitud 10.
longitud	len_geometric	Geom($p = 0,2$), recortada a $[3, 10]$.
longitud	len_poisson	Poisson($\lambda = 5$), recortada a $[3, 10]$.
longitud	len_extrapolate_long	$U\{11, \dots, 20\}$, fuera del soporte.
conjunta	joint_high_long	Gaussiana alta $\times$ todas-largas.
conjunta	joint_low_short	Gaussiana baja $\times$ todas-cortas.
conjunta	joint_uniform_geom	Uniforme $\times$ geométrica.
conjunta	joint_extrap_both	Severidad y longitud fuera del soporte.
estructural	struct_few_long_blocks	4 bloques $\times$ 16 secuencias.
estructural	struct_many_short_blocks	16 bloques $\times$ 4 secuencias.
estructural	struct_more_total	8 bloques $\times$ 16 secuencias ($n = 128$).

4.8. Protocolo de entrenamiento y evaluación

1. **Optimización bayesiana de hiperparámetros.** Ensayos Optuna/TPE ejecutados por paquete: **pes_q1** (100), **pes_dq1** (100), **pes_a2c** (100), **pes_dqn** (47), **pes_rdn** (20) y **pes_trf** (10). El número de ensayos se fija por línea de comandos.
2. **Reentrenamiento final.** Con el mejor conjunto de hiperparámetros θ^* y la semilla de su ensayo, cada modelo optimizado se reentrena desde cero con el presupuesto de episodios del mejor ensayo: **pes_q1** (550 000), **pes_dq1** (360 000), **pes_a2c** (125 000), **pes_dqn** (40 000) y **pes_rdn/pes_trf** (30 000 cada uno). **pes_base** entrena con parámetros fijos durante 1 000 000 de episodios.
3. **Evaluación determinista.** Se evalúan las $n = 64$ secuencias con la política

codiciosa ($\varepsilon = 0$), es decir, seleccionando en cada paso la acción factible de mayor valor.

4. **Análisis estadístico.** Para cada modelo se calculan las matrices modelo $\times$ escenario de media, desviación estándar, degradación, p de Welch [26], d de Cohen [25] y KL de acciones frente a la referencia. Para cada par de modelos de una misma suite se calculan, sobre los 21 escenarios de perturbación, la d de Cohen, el p de Welch y la KL simetrizada entre histogramas de rendimiento (Cap. 5). Como todos los modelos se evalúan sobre las mismas secuencias, la prueba de Welch se interpreta como un contraste de referencia entre distribuciones; una inferencia estrictamente pareada requeriría analizar las diferencias por secuencia.

La reproducibilidad del protocolo se basa en la semilla 42 para la generación de las secuencias y en su reutilización para todos los modelos. Los resultados se almacenan en archivos versionados por fecha. Las versiones del entorno son Python 3.12, TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17.

5. Resultados

Este capítulo presenta los resultados empíricos del benchmark mPES, comparando dos suites de **seis modelos** sobre $n = 64$ secuencias por escenario, agrupadas en **cuatro familias** — *severidad*, *longitud*, *conjunta* y *estructural* — que cubren las 22 condiciones del catálogo `general/`. La métrica principal es el *desempeño normalizado medio* $\bar{r} \in [0, 1]$ (registrado en los JSON como `global_mean_perf`), definida como el promedio sobre las n secuencias de la métrica por secuencia de la Ecuación (4.15):

$$\bar{r} = \frac{1}{n} \sum_{i=1}^n \frac{S_{\text{peor}}^{(i)} - S_{\text{agente}}^{(i)}}{S_{\text{peor}}^{(i)} - S_{\text{mejor}}^{(i)}}, \quad (5.1)$$

donde, para la secuencia i , $S_{\text{agente}}^{(i)}$ es la severidad acumulada obtenida por el agente, $S_{\text{peor}}^{(i)}$ la que resulta de no asignar recursos y $S_{\text{mejor}}^{(i)}$ la mínima alcanzable bajo el presupuesto de la secuencia. Por construcción cada término está en $[0, 1]$: 0 equivale a no actuar y 1 a alcanzar el óptimo factible. A diferencia de la recompensa del MDP (Ecuación (4.3)), esta métrica es adimensional y está normalizada por las cotas de cada secuencia, por lo que

resulta comparable entre escenarios de distinta severidad y longitud y entre arquitecturas con distinto *shaping* de recompensa.

5.1. Familias de perturbaciones evaluadas

El catálogo completo de las 22 condiciones se detalla en la Sec. 4.7 (Tabla 4.2); las cuatro familias agrupan, respectivamente, 9 escenarios de *severidad*, 5 de *longitud*, 4 *conjuntos* y 3 *estructurales*, que junto con la condición de referencia suman 22. Tres de ellos (`sev_extrapolate_high`, `len_extrapolate_long` y `joint_extrap_both`) utilizan valores fuera del soporte de entrenamiento. La condición `sev_base` actúa como referencia de cada modelo: las métricas derivadas de este capítulo — degradación $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$, divergencia KL entre distribuciones de acciones, d de Cohen y p de Welch — se calculan siempre respecto de esa fila. Cada celda del benchmark corresponde a un par (modelo, escenario) ejecutado en inferencia sobre el mismo conjunto de 64 secuencias generadas con semilla 42.

5.2. Suite individual: resumen cuantitativo

La Tabla 5.1 resume la condición de referencia de los seis modelos individuales. El Transformer causal obtiene el mayor desempeño medio (0,927) y la menor dispersión entre secuencias (0,045). En los 21 escenarios de perturbación su media es 0,930, con degradación media $-0,002$, es decir, un rendimiento ligeramente superior al de la referencia en promedio. El Transformer es, por tanto, el mejor modelo individual. La suite de ensambles se presenta por separado en la Sec. 5.9.

Cuadro 5.1: Desempeño normalizado medio $\bar{r}$ (Ec. (5.1)) y desviación estándar entre secuencias σ de cada modelo individual en la condición de referencia **sev_base** ($n = 64$, semilla = 42). **pes_base** se incluye como referencia sin optimización.

Modelo individual	$\bar{r}$	σ
pes_trf	0,927	0,045
pes_rdq	0,899	0,049
pes_dql	0,896	0,048
pes_dqn	0,894	0,055
pes_a2c	0,887	0,063
pes_ql	0,887	0,061
pes_base	0,871	0,074

5.3. Reducción de la severidad normalizada

Tomando **pes_ql** como línea de base tabular y utilizando $1 - \bar{r}$ como medida de severidad residual normalizada, la reducción relativa de un modelo m en la condición de referencia es

$$\Delta_{\text{rel}}(m) = \frac{(1 - \bar{r}_{\text{ql}}) - (1 - \bar{r}_m)}{1 - \bar{r}_{\text{ql}}} = \frac{\bar{r}_m - \bar{r}_{\text{ql}}}{1 - \bar{r}_{\text{ql}}}. \quad (5.2)$$

Con $\bar{r}_{\text{ql}} = 0,887$, la severidad residual de la línea de base es 0,113. Para **pes_trf** ($\bar{r} = 0,927$) la residual es 0,073 y $\Delta_{\text{rel}} \approx 0,35$. Es decir, el Transformer reduce la severidad residual normalizada en torno a un 35 % respecto de la línea de base tabular. La cifra es descriptiva de la condición de referencia y no se extiende a los escenarios de perturbación.

La hipótesis H1a (Sec. 1.3) queda respaldada por el ordenamiento de la suite individual. La Tabla 5.2 resume los contrastes pareados de mayor interés entre modelos individuales, calculados sobre los 21 escenarios de perturbación (21×64 observaciones por modelo). Los valores p son extremadamente pequeños porque la muestra es grande y las observaciones de distintos escenarios se agrupan; deben leerse junto con la d de Cohen, que mide la magnitud de la diferencia en unidades de desviación estándar, y no como prueba de un mecanismo causal.

Cuadro 5.2: Contrastes pareados de la suite individual sobre los 21 escenarios de perturbación. d : tamaño de efecto de Cohen (positivo cuando el primer modelo tiene mayor media); $\log_{10} p$: valor p bilateral de Welch en escala logarítmica; D_{KL}^{sym} : divergencia simetrizada entre los histogramas de rendimiento de ambos modelos.

Comparación	d de Cohen	$\log_{10} p$	D_{KL}^{sym}
pes_trf vs pes_q1	0,78	−87,8	0,49
pes_trf vs pes_rdqn	0,57	−48,9	0,34
pes_trf vs pes_dqn	0,51	−39,2	0,17

5.4. Comportamiento por familia algorítmica

Las observaciones siguientes se apoyan en la matriz de desempeño por escenario (Figura 5.1) y en el informe de la suite.

Tabular (pes_base, pes_q1, pes_dq1). Su desempeño es comparable al de los modelos profundos en la condición de referencia y en secuencias cortas (len_all_short: 0,943 para pes_q1). El peor escenario de los tres es la extrapolación de severidad (sev_extrapolate_high: 0,627, 0,762 y 0,785 respectivamente), donde la tabla Q sólo cubre severidades 0–9 y la inferencia recorta el índice de estado al último nivel, de modo que $S \in \{10, 11, 12\}$ se evalúa con la política aprendida para $S = 9$. También pierden rendimiento con secuencias largas (len_all_long: 0,859 y 0,878 para pes_q1 y pes_dq1).

pes_dqn. La aproximación funcional recibe la severidad normalizada $S/9$ como entrada continua, también cuando excede 1. En sev_extrapolate_high obtiene 0,890 frente a 0,762 de pes_q1; la diferencia es descriptiva y no aísla el efecto de la representación de otros factores de entrenamiento. Su peor escenario es len_extrapolate_long (0,841).

pes_a2c. Obtiene su mejor resultado en joint_low_short (0,988) y joint_extrap_both (0,949), y el peor en len_extrapolate_long (0,826). Su media bajo perturbación (0,896) supera a su referencia (0,887).

pes_rdqn. La capa LSTM sobre la ventana de seis estados sitúa al modelo en 0,899 en la referencia, por encima de pes_dqn. En len_extrapolate_long alcanza 0,889 frente a 0,841 de pes_dqn, mientras que en sev_extrapolate_high desciende a 0,831, su peor escenario.

`pes_trf`. Obtiene la mayor media en 16 de los 22 escenarios de la suite individual.

En `sev_extrapolate_high` alcanza 0,996 y en `joint_extrap_both` 0,997; su peor escenario es `len_extrapolate_long` (0,860). Estos resultados son compatibles con la hipótesis de que la ventana de estados y la atención causal aportan información útil sobre la tendencia de la severidad, pero el diseño experimental no permite aislar ese mecanismo (véase Sec. 6).

5.5. Artefactos generados por el procedimiento experimental

El módulo `general/scripts/benchmark.py`, junto con `analysis.py` y `figures.py`, genera para cada suite (`individual` y `ensemble`) los siguientes artefactos en `general/results/<suite>/`:

- `cells/<modelo>__<escenario>.json`: rendimiento por secuencia, distribución empírica de acciones y estadísticos de cada celda.
- `matrices/*.csv`: matrices $\text{modelo} \times \text{escenario}$ de media, desviación estándar, degradación, p y $\log_{10} p$ de Welch, d de Cohen y KL de acciones frente a la referencia.
- `comparison_metrics.json`: contrastes pareados entre modelos de la suite.
- `figures/`: mapas de calor, curvas por familia y por escenario, ranking y perfiles de generalización.
- `report.md`: resumen ejecutivo de la suite.

Las matrices principales de la suite individual se representan en las Figuras 5.1–5.4: desempeño medio, degradación frente a la referencia, $\log_{10} p$ de Welch y divergencia KL de acciones. La Figura 5.5 muestra los tamaños de efecto pareados entre los modelos individuales. En los mapas de degradación, las tres columnas estructurales muestran valor nulo por construcción, ya que reproducen la distribución de referencia con otra partición en bloques. Los mapas equivalentes de la suite de ensambles se presentan en la Sec. 5.9.

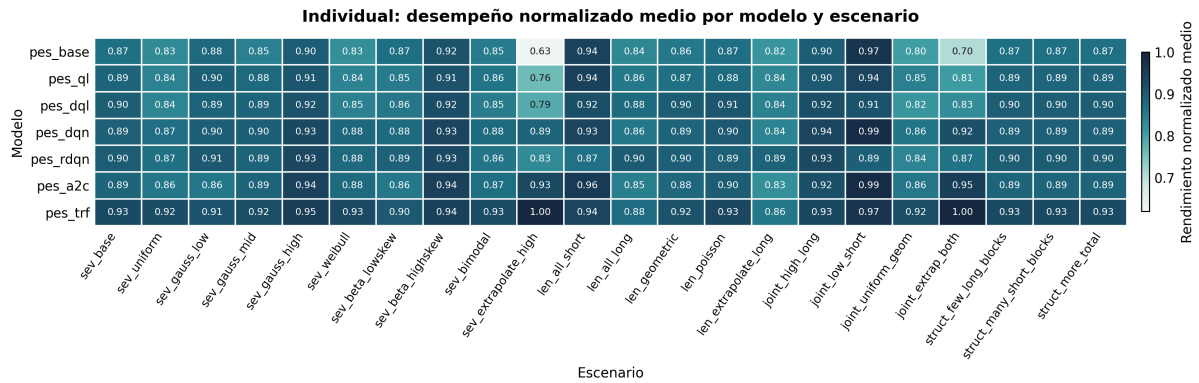


Figura 5.1: Suite individual: desempeño normalizado medio $\bar{r}$ por modelo y escenario. Cada celda promedia 64 secuencias; la primera columna es la condición de referencia **sev_base**.

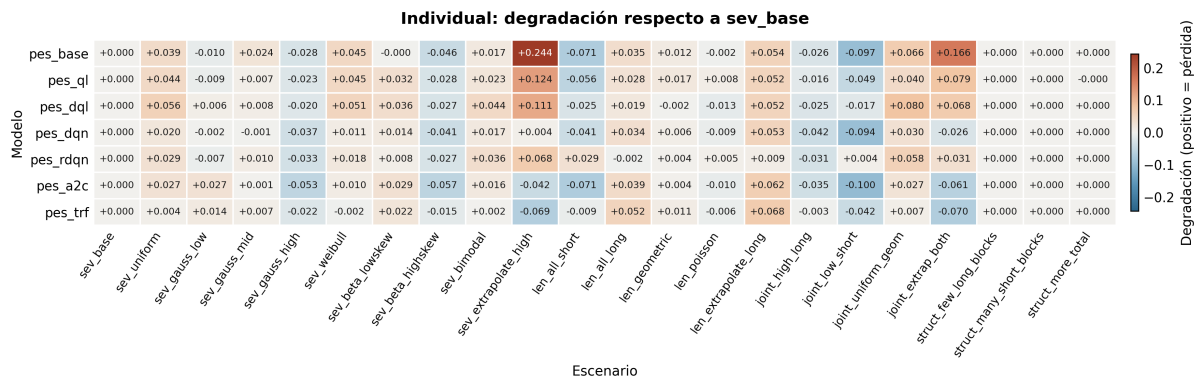


Figura 5.2: Suite individual: degradación $\Delta\bar{r} = \bar{r}_{\text{ref}} - \bar{r}_{\text{celda}}$ respecto de **sev_base**. Los valores positivos indican pérdida de desempeño y los negativos, mejora.

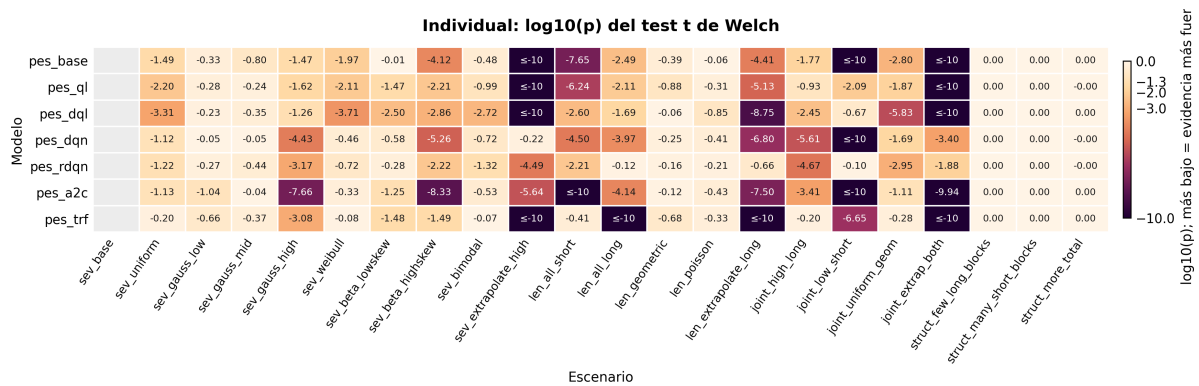


Figura 5.3: Suite individual: $\log_{10} p$ de la prueba t de Welch entre los 64 valores por secuencia de cada celda y los de la referencia del mismo modelo. Valores más negativos indican mayor evidencia de cambio en la media; las celdas con p inferior al límite de la escala se marcan como recortadas.

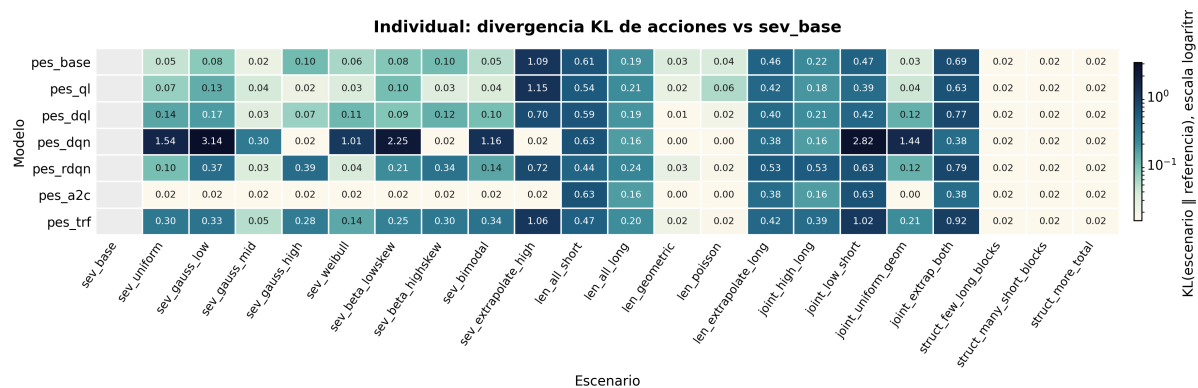


Figura 5.4: Suite individual: divergencia D_{KL} (Ecuación (2.21)) entre la distribución empírica de acciones de cada escenario y la de la referencia del mismo modelo. Mide el desplazamiento de la política, no el rendimiento.

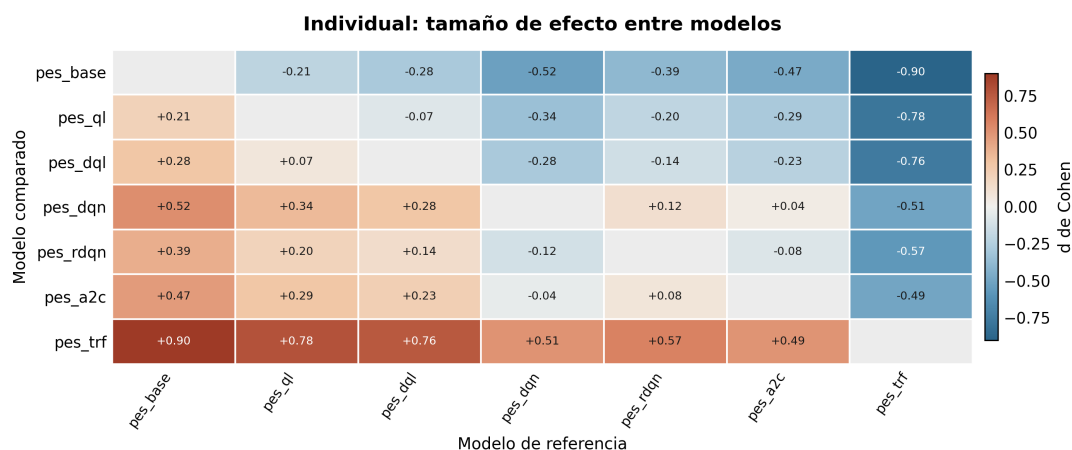


Figura 5.5: d de Cohen (Ecuación (2.19)) entre pares de modelos de la suite individual, calculada sobre los 21 escenarios de perturbación. Un valor positivo indica que el modelo de la fila supera en media al de la columna.

5.6. Resultados por modelo

Las Figuras 5.6–5.12 muestran el desempeño por secuencia de los siete paquetes individuales en la condición de referencia. En consonancia con la Tabla 5.1, los métodos tabulares (Figs. 5.6–5.8) presentan medias más bajas y mayor dispersión entre secuencias (σ entre 0,048 y 0,074), mientras que `pes_trf` (Fig. 5.12) combina la mayor media con la menor dispersión (0,045).

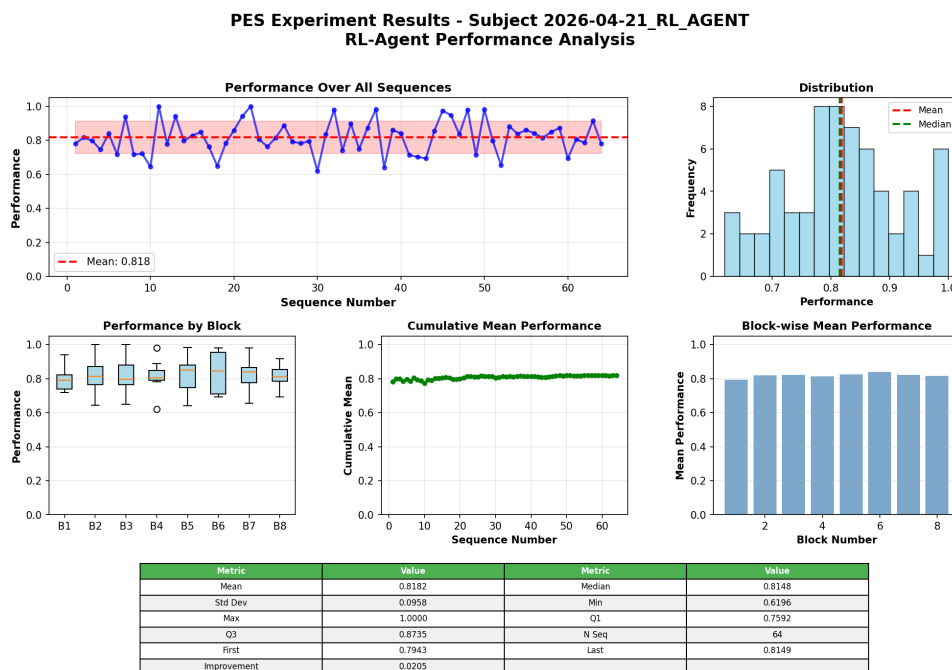


Figura 5.6: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de `pes_base`.

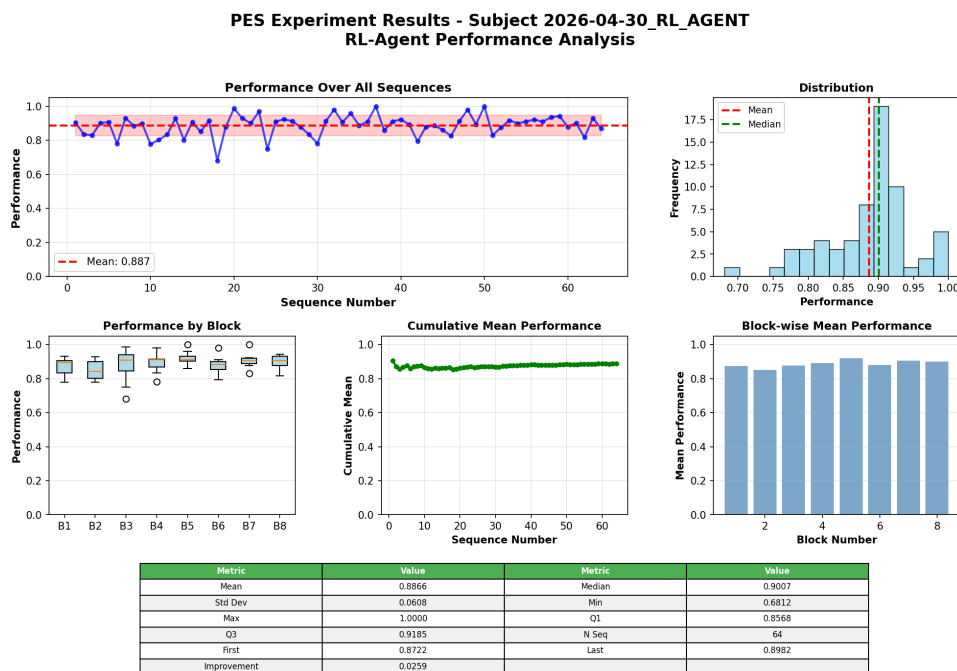


Figura 5.7: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_q1.

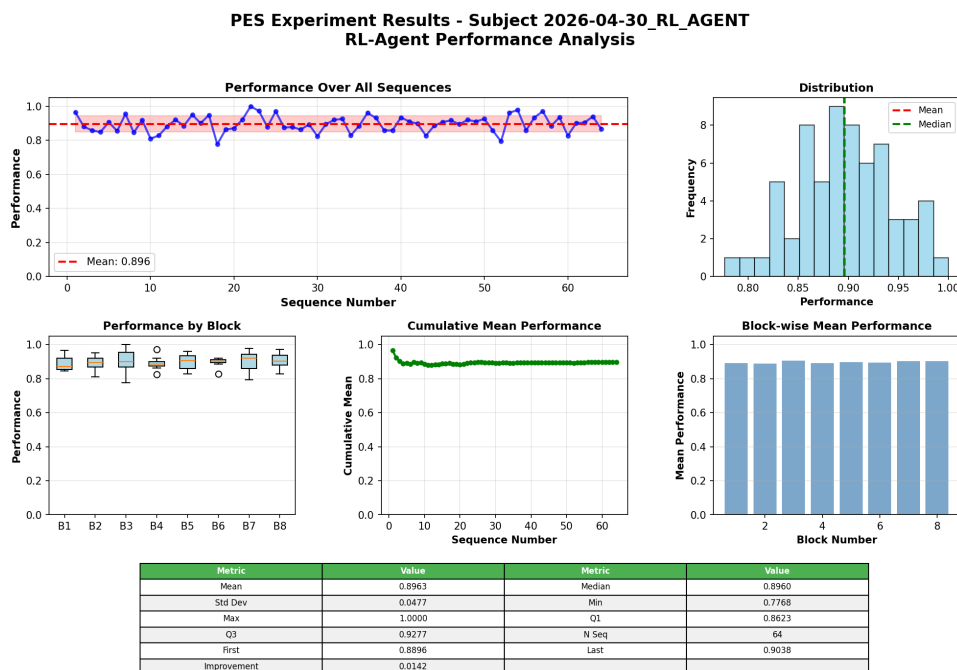


Figura 5.8: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dq1.

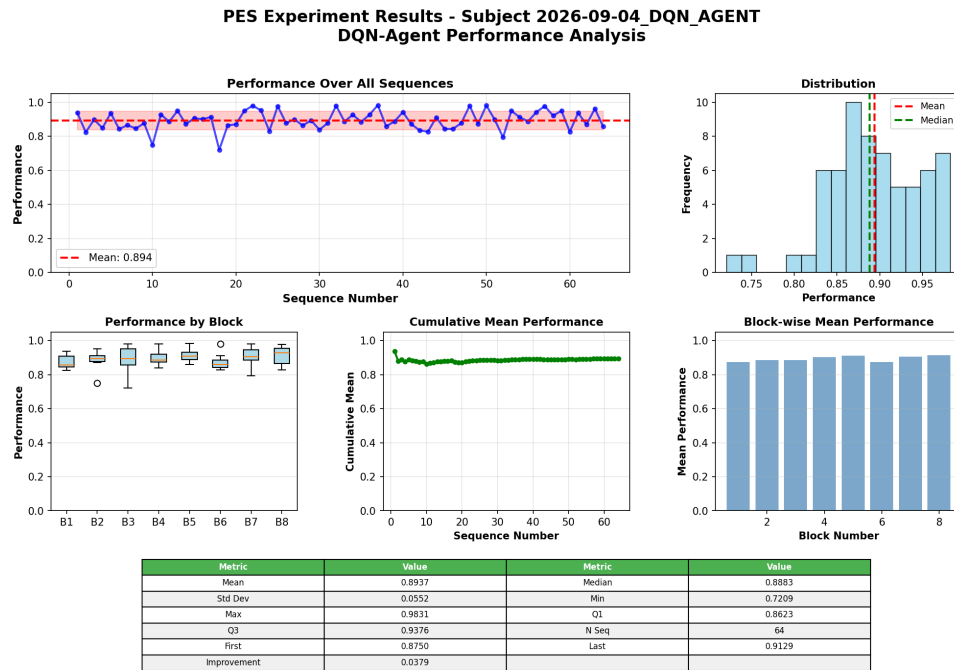


Figura 5.9: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_dqn.

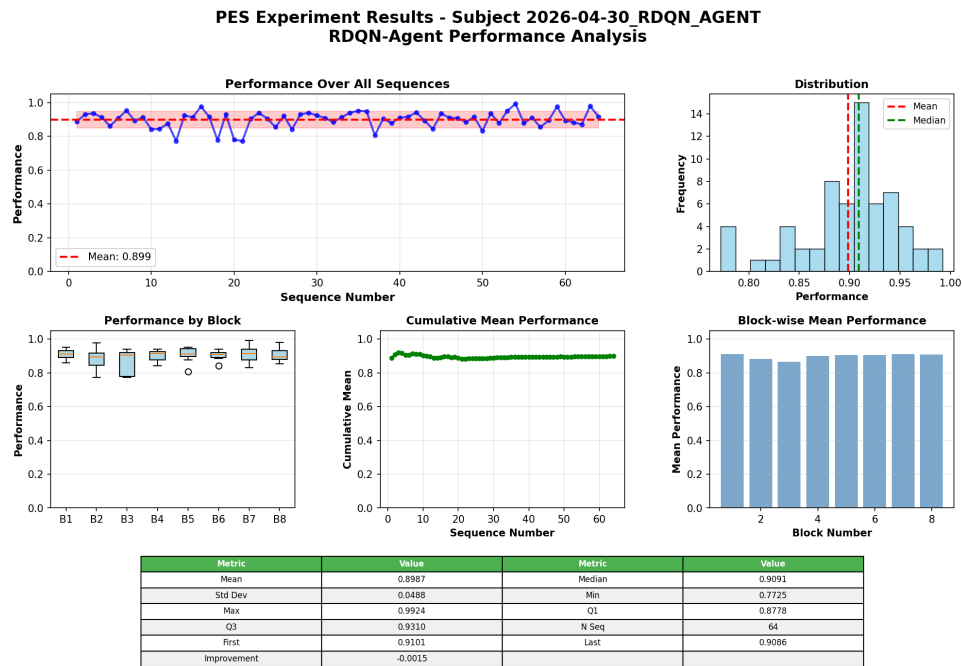


Figura 5.10: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_rdqn.

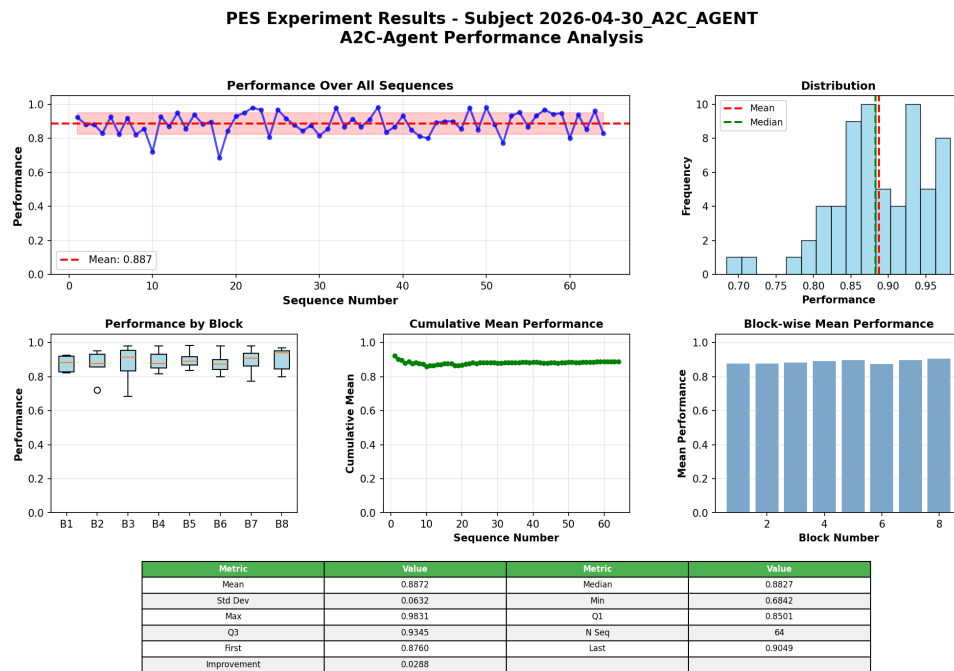


Figura 5.11: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_a2c.

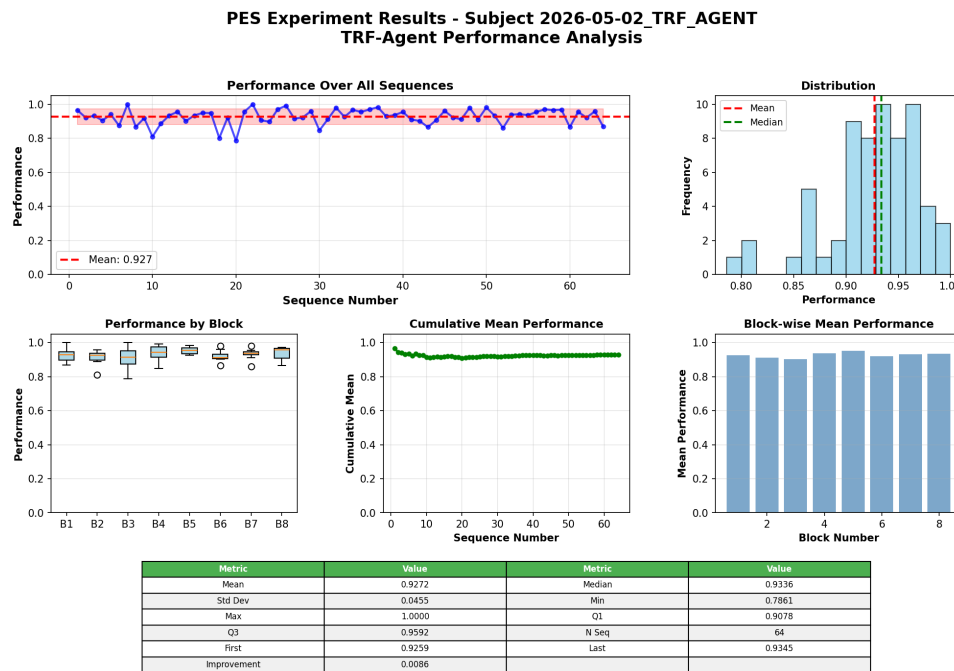


Figura 5.12: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de pes_trf.

5.7. Resultados por escenario

La Figura 5.13 presenta curvas de desempeño por secuencia en seis escenarios representativos, organizadas por familia de perturbación: severidad (`sev_bimodal`, `sev_gauss_high`, `sev_beta_highskew`), longitud (`len_poisson`, `len_extrapolate_long`) y conjunta (`joint_high_long`). En cada panel las 64 secuencias se ordenan de menor a mayor $\bar{r}$, de modo que una curva situada por encima de otra indica mejor desempeño en toda la distribución de secuencias, y la pendiente inicial refleja la magnitud de los peores casos. La Figura 5.14 resume, para la suite individual, la media en la referencia y la media en los 21 escenarios de perturbación.

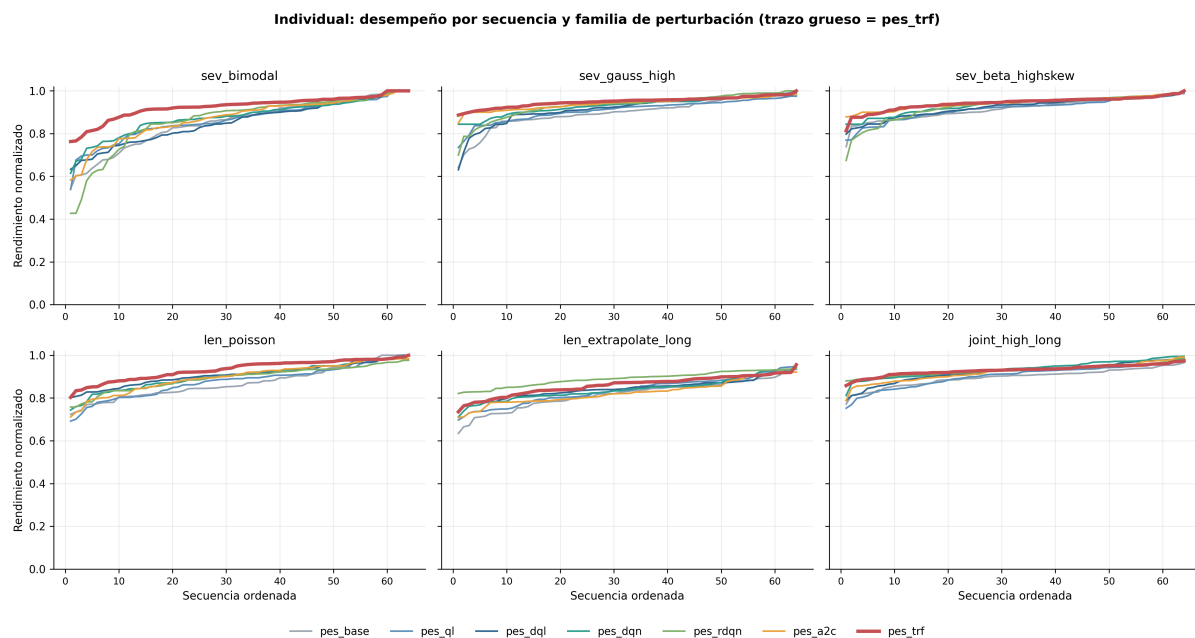


Figura 5.13: Suite individual: desempeño por secuencia ordenado de menor a mayor $\bar{r}$ (Ec. (5.1)) en seis escenarios. Fila superior: severidad; fila inferior: longitud y conjunta. Se incluyen los siete paquetes individuales, con `pes_base` como referencia sin optimización; el trazo grueso corresponde a `pes_trf`, el modelo de mayor media bajo perturbación.

5.8. Evaluación en escenarios de extrapolación

El catálogo incluye tres condiciones cuyos valores quedan fuera del soporte de entrenamiento: `sev_extrapolate_high`, `joint_extrap_both` y `len_extrapolate_long`. La Figura 5.15 muestra el desempeño por secuencia de `pes_trf` en cada una, junto con el modelo tabular de mayor rendimiento (`pes_dql`) en la extrapolación de severidad, el modelo de gradiente de política (`pes_a2c`) en la extrapolación conjunta y `pes_dqn` en la extrapolación de longitud. En los dos primeros escenarios las curvas son casi planas

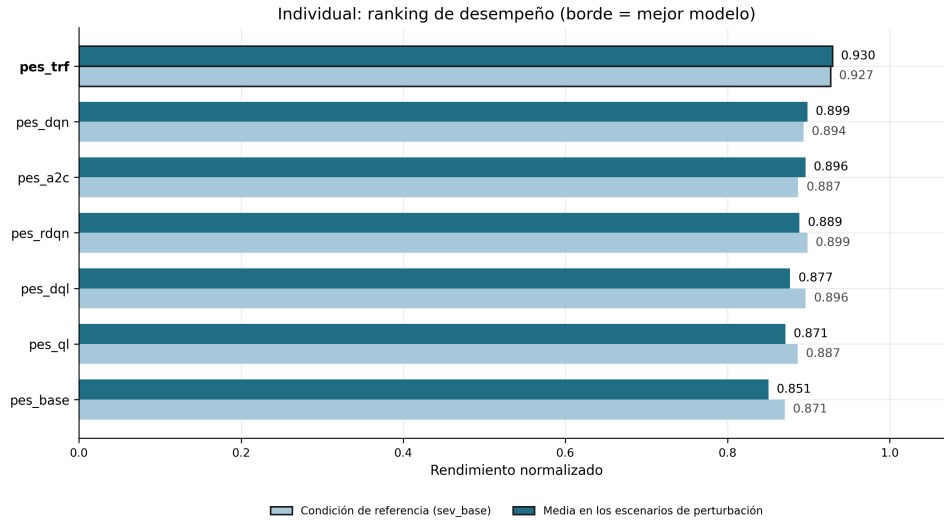


Figura 5.14: Media de $\bar{r}$ en la condición de referencia (barra clara) y en los 21 escenarios de perturbación (barra oscura) para la suite individual; el borde destaca a **pes_trf**.

y cercanas a 1 para **pes_trf**, lo que indica que la asignación elegida alcanza la cota factible en la mayoría de las secuencias; en la extrapolación de longitud la dispersión entre secuencias es mayor para todos los modelos. Las curvas describen el comportamiento observado y no se interpretan como prueba de un mecanismo causal.

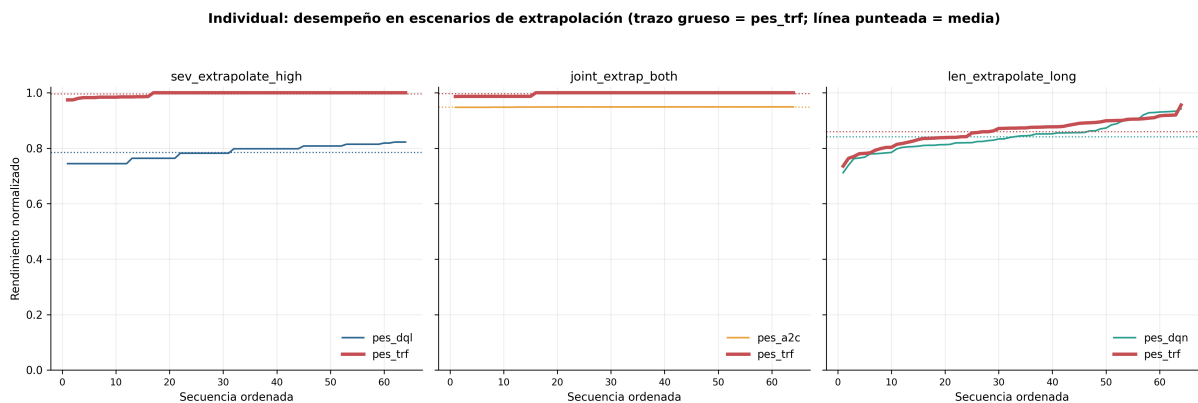


Figura 5.15: Desempeño por secuencia, ordenado de menor a mayor $\bar{r}$, en los tres escenarios fuera del soporte de entrenamiento. *Izquierda:* sev_extrapolate_high (pes_dql y pes_trf). *Centro:* joint_extrap_both (pes_a2c y pes_trf). *Derecha:* len_extrapolate_long (pes_dqn y pes_trf). El trazo grueso corresponde a **pes_trf** y las líneas punteadas marcan la media de cada modelo sobre las 64 secuencias.

5.9. Suite de ensambles

Las seis variantes de ensamble se evalúan como una suite independiente sobre los mismos 22 escenarios y las mismas 64 secuencias por celda. La Tabla 5.3 recoge la comparación. **pes_ens** obtiene la mayor media bajo perturbación (0,939) y la menor degradación máxima (0,037), seguido por **pes_ens_trf_guard** (0,931) y **pes_ens_consensus_prior** (0,923). La

variante de consenso sin prior alcanza 0,904, mientras que **pes_ens_sprb** y **pes_ens_accq** obtienen 0,902 y 0,901, respectivamente, y concentran su mayor degradación en la extrapolación de severidad.

Cuadro 5.3: Rendimiento en la condición de referencia, media en los 21 escenarios de perturbación y mayor degradación observada para las seis variantes de ensamble.

Ensamble	Referencia	Media bajo perturb.	Mayor degradación
pes_ens	0,937	0,939	0,037
pes_ens_trf_guard	0,928	0,931	0,067
pes_ens_consensus_prior	0,918	0,923	0,042
pes_ens_consensus	0,893	0,904	0,063
pes_ens_sprb	0,914	0,902	0,054
pes_ens_accq	0,914	0,901	0,055

Con la misma línea de base tabular de la Ecuación (5.2), **pes_ens** ($\bar{r} = 0,937$) deja una severidad residual de 0,063 y $\Delta_{\text{rel}} \approx 0,45$, frente al 0,35 del Transformer individual. El mejor resultado del benchmark completo corresponde, por tanto, a **pes_ens**: su media bajo perturbación (0,939) supera a la de **pes_trf** (0,930). Este resultado no sostiene una superioridad del Transformer frente a todos los sistemas evaluados, sino sólo frente a los modelos individuales (Sec. 5.2).

La Figura 5.16 muestra este ordenamiento al comparar, para cada variante, la media en la referencia con la media en los 21 escenarios de perturbación. La Figura 5.17 permite observar la dispersión por secuencia y por familia. En ella, **pes_ens** conserva una banda estrecha y un rendimiento alto, mientras que **pes_ens_sprb** y **pes_ens_accq** presentan mayor dispersión en los escenarios de severidad.

La Figura 5.18 concentra la lectura en los tres escenarios cuyos valores quedan fuera del soporte de entrenamiento. El ensamble **pes_ens** alcanza 1,000 en la extrapolación de severidad y 0,900 en la de longitud; por el contrario, **pes_ens_trf_guard** y **pes_ens_consensus** alcanzan sus mayores pérdidas en **len_extrapolate_long**, con degradaciones 0,067 y 0,063 respectivamente.

Los mapas de las Figuras 5.19, 5.20 y 5.21 complementan el ranking con tres tipos de evidencia. El mapa de desempeño permite comparar las seis variantes escenario por

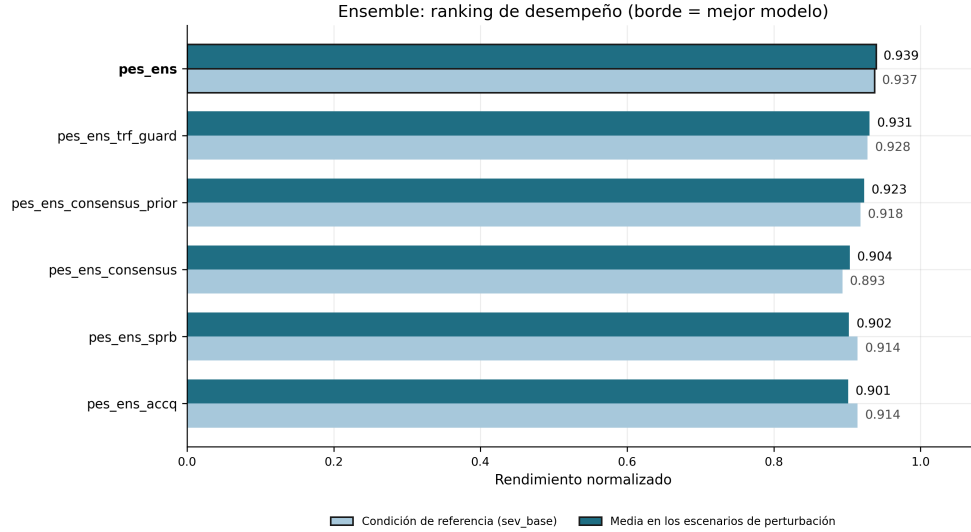


Figura 5.16: Media del desempeño normalizado en la condición de referencia y en los 21 escenarios de perturbación para las seis variantes de ensemble; el borde destaca a **pes_ens**.

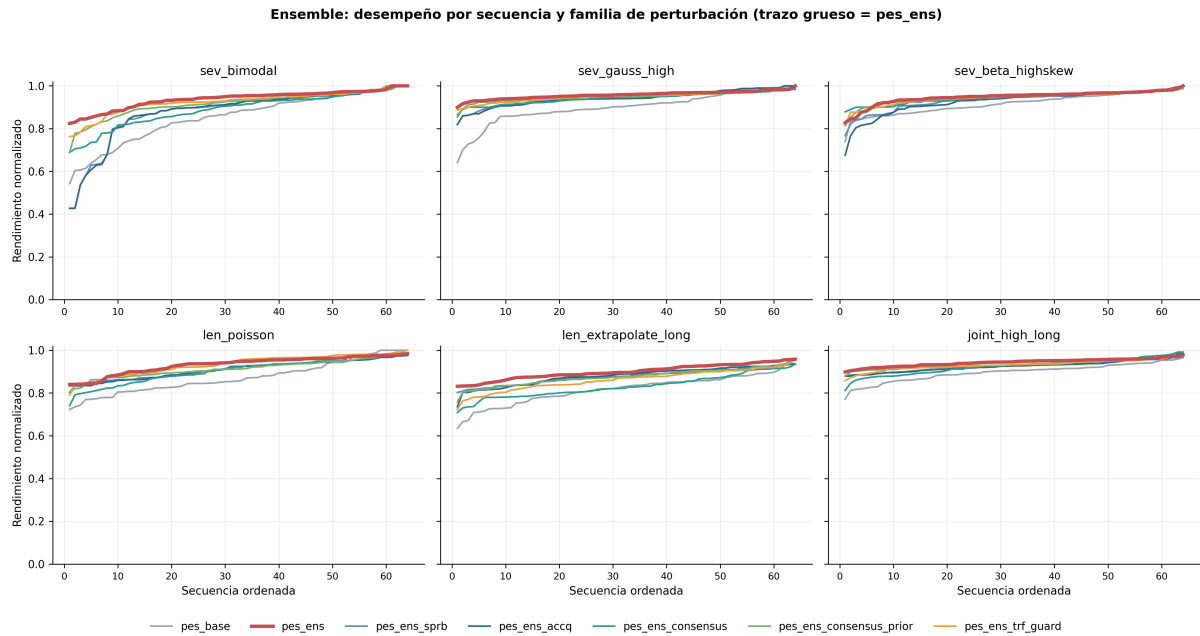


Figura 5.17: Curvas de desempeño por secuencia, ordenadas de menor a mayor, para los seis ensambles en escenarios de severidad, longitud y perturbación conjunta. El trazo grueso corresponde a **pes_ens**, la variante de mayor media bajo perturbación.

escenario; el de degradación muestra que la longitud es el factor más desfavorable para **pes_ens_trf_guard**, **pes_ens_consensus** y **pes_ens_consensus_prior**; y los mapas de Welch y d permiten separar diferencias de media de su magnitud estandarizada. La Tabla 5.4 y la Figura 5.21 muestran que **pes_ens** supera a **pes_ens_consensus** con $d = 0,62$ y $\log_{10} p = -55,6$, mientras que la diferencia frente a **pes_ens_trf_guard** es pequeña ($d = 0,17$, $\log_{10} p = -5,1$). El valor p indica evidencia estadística de una diferencia de medias, pero no identifica por sí solo la causa de la diferencia ni su relevancia

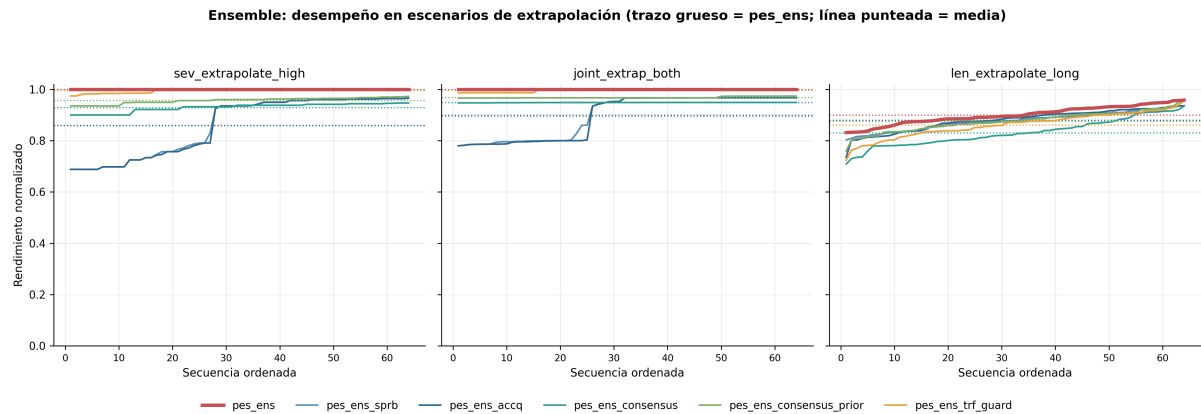


Figura 5.18: Curvas por secuencia de las seis variantes de ensemble en los tres escenarios de extrapolación; el trazo grueso corresponde a **pes_ens** y las líneas punteadas a la media de cada variante. Las curvas muestran la distribución de rendimiento y no constituyen una prueba causal sobre la regla de combinación.

operativa.

Cuadro 5.4: Contrastes pareados de la suite de ensambles sobre los 21 escenarios de perturbación; columnas como en la Tabla 5.2.

Comparación	d de Cohen	$\log_{10} p$	D_{KL}^{sym}
pes_ens vs pes_ens_consensus	0,62	-55,6	0,25
pes_ens vs pes_ens_trf_guard	0,17	-5,1	0,03

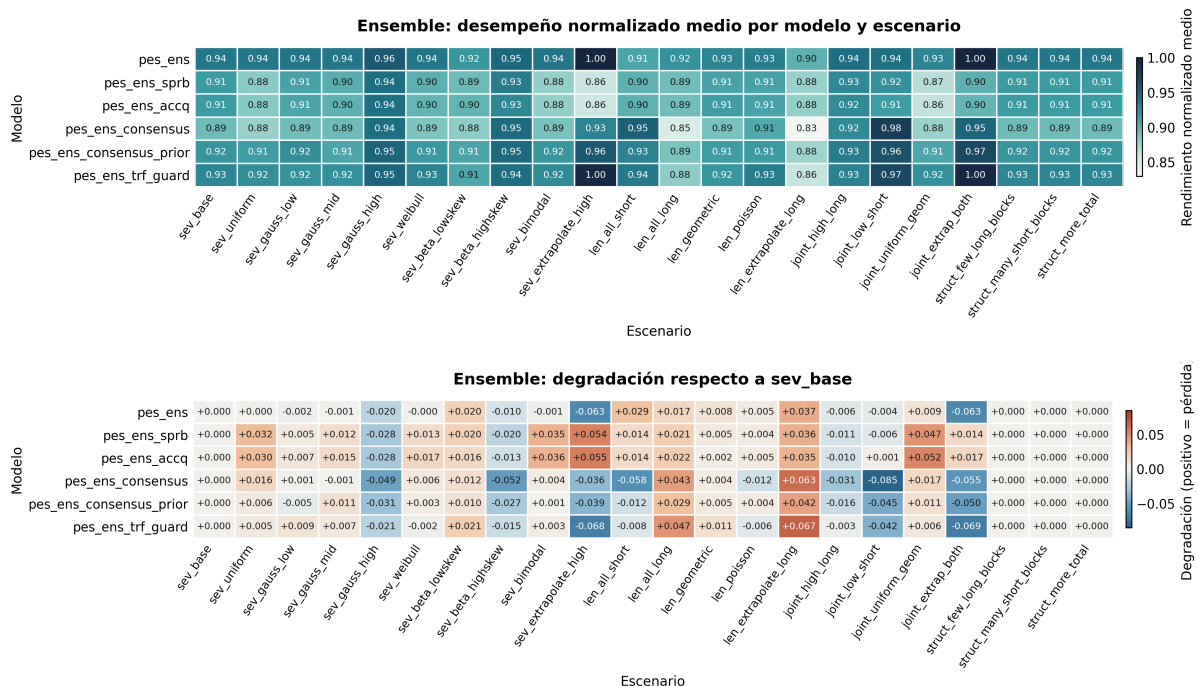


Figura 5.19: Suite de ensambles: desempeño normalizado medio por escenario (arriba) y degradación respecto de la referencia (abajo) para las seis variantes de ensemble.

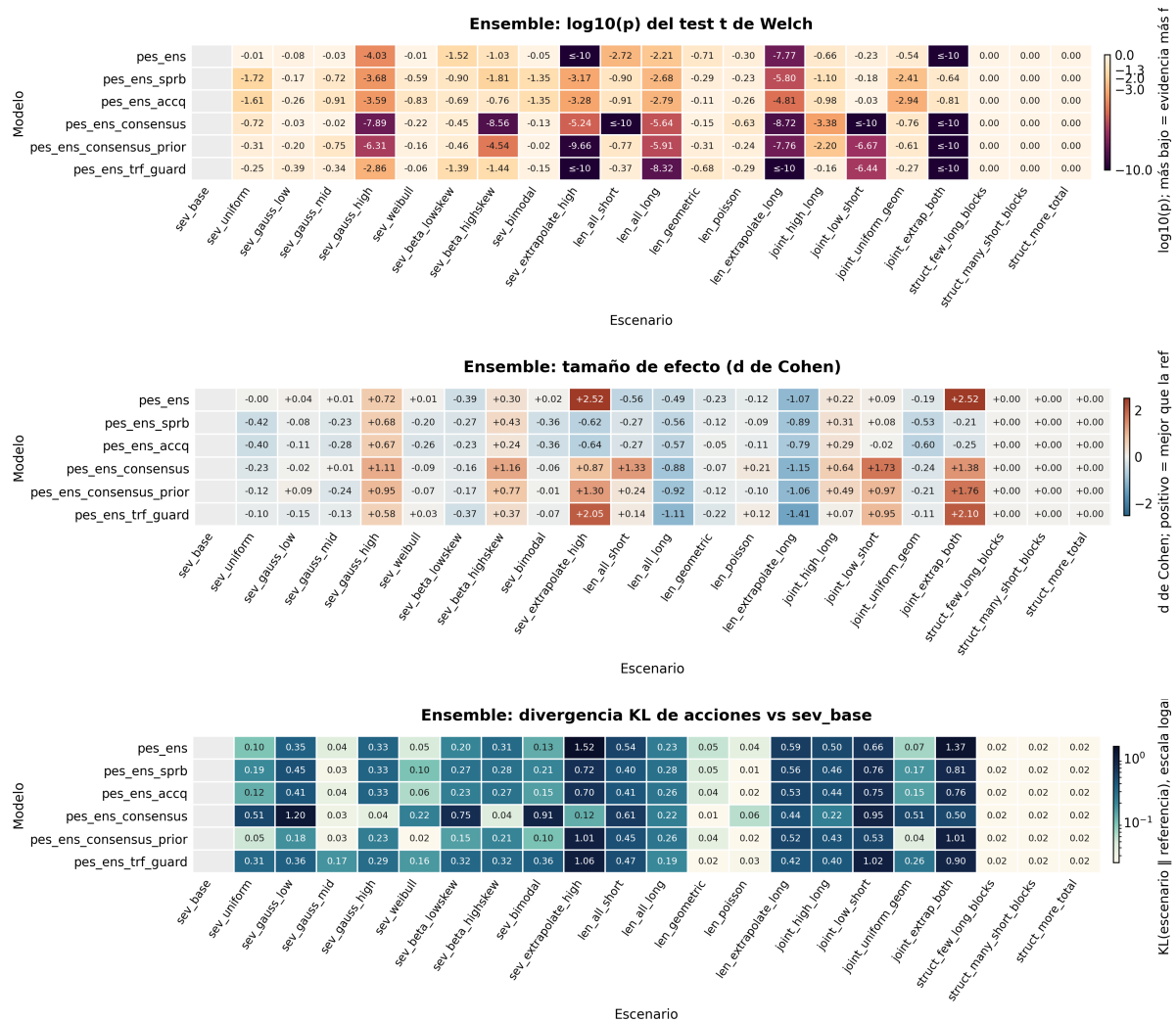


Figura 5.20: Suite de ensambles: $\log_{10} p$ de Welch (arriba), d de Cohen (centro) y divergencia KL de acciones (abajo) por modelo y escenario. Welch cuantifica evidencia de diferencia de medias frente a la referencia; Cohen incorpora la magnitud estandarizada; KL cuantifica el desplazamiento de la distribución de acciones. Ninguna de las tres medidas identifica por sí sola un mecanismo causal.

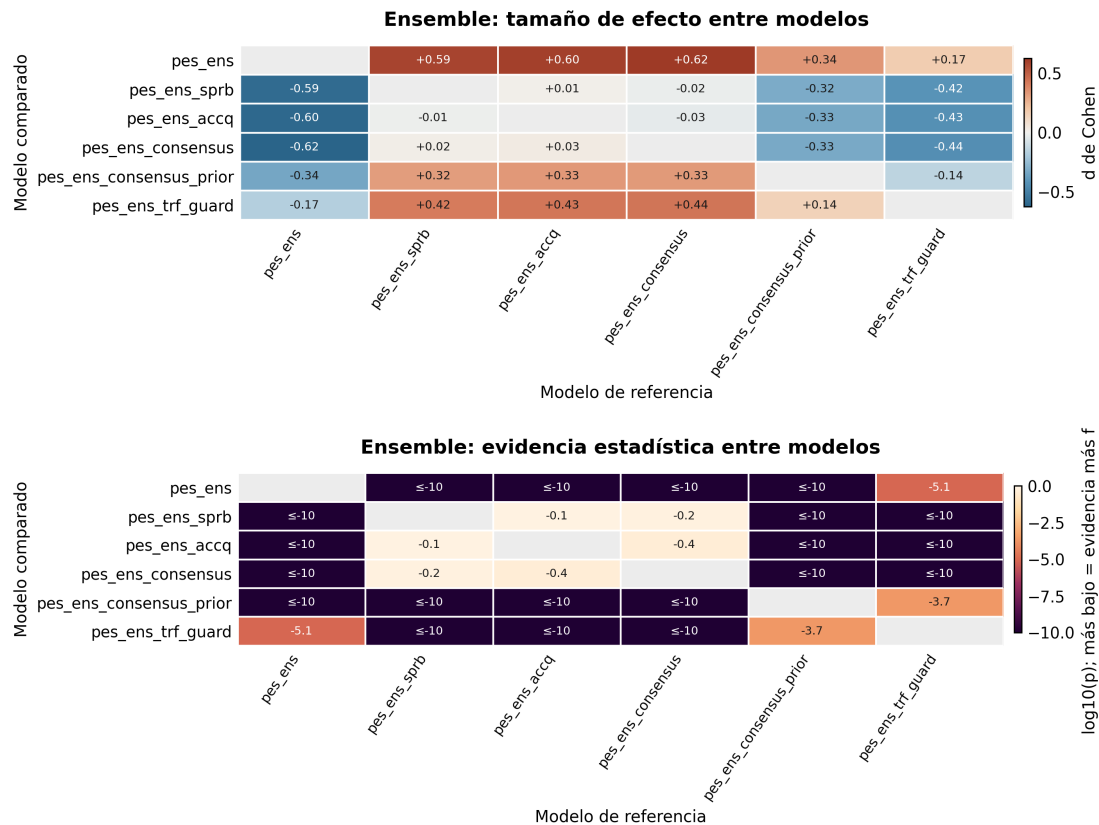


Figura 5.21: Contrastes pareados entre las seis variantes de ensamble: d de Cohen arriba y $\log_{10} p$ de Welch abajo, calculados sobre los 21 escenarios de perturbación comunes y sus 64 secuencias.

La Figura 5.22 presenta el desempeño por secuencia de `pes_ens` en la condición de referencia, con el mismo formato que las figuras individuales de la Sec. 5.6: el ensamble combina la mayor media del benchmark con la menor dispersión entre secuencias ($\sigma = 0,035$).

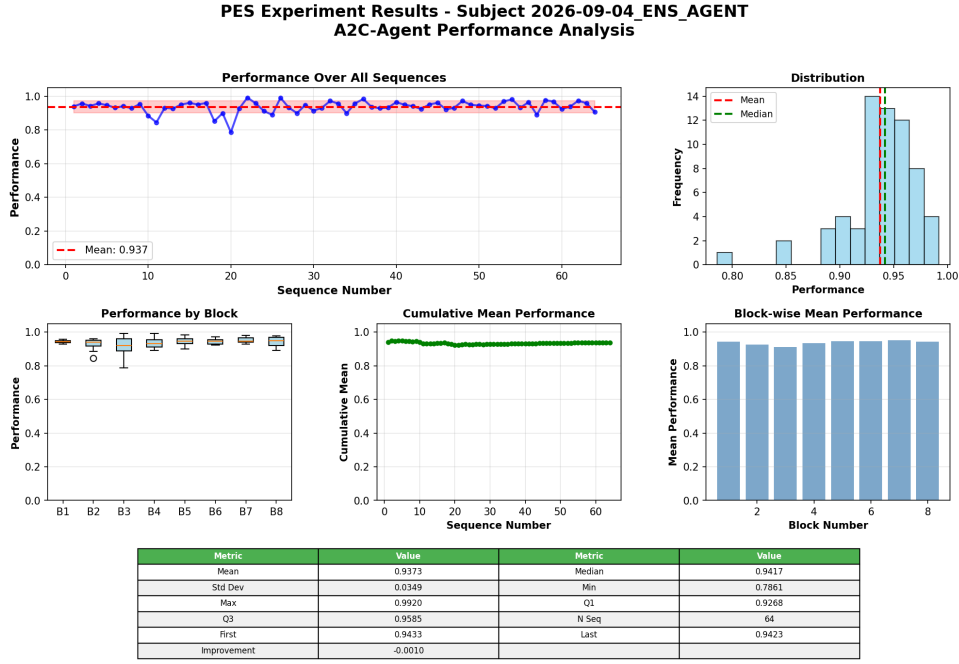


Figura 5.22: Desempeño por secuencia, distribución, estadísticos por bloque y resumen descriptivo de `pes_ens`.

5.10. Análisis de cuantificación de incertidumbre

La medida de confianza $1 - H_{\text{norm}}$ interviene en dos lugares del trabajo. Como traza por secuencia se registra únicamente para `pes_trf`, que actúa como estudio de caso. Como factor de decisión conjunta se calcula, en cada paso, para cada miembro de los seis ensambles: `pes_ens` multiplica el peso base de cada miembro por $0,1 + c_k$, y las cinco variantes de la Sec. 4 usan $\tilde{w}_k = w_k c_k^\rho$ con $\rho = 1$, siendo $c_k = 1 - H_{\text{norm}}(p_k)$ la confianza del miembro k sobre las acciones factibles. Los artefactos del benchmark no almacenan estos valores c_k por paso, de modo que su efecto sólo puede observarse de forma agregada en el rendimiento de cada ensamble (Tabla 5.3).

Fuera de los ensambles, las salidas de los agentes no son comparables entre sí como probabilidades: las tablas Q y las redes DQN, RDQN y Transformer producen valores cuya escala depende de la parametrización y del entrenamiento, y el actor de A2C produce directamente una distribución de probabilidad. Convertirlas en probabilidades comparables exigiría un procedimiento de calibración y una validación específica de su calidad probabilística, que no forman parte de este trabajo.

El código de inferencia de `pes_trf` calcula la confianza a partir de la entropía de los valores Q de las acciones factibles, después de desplazarlos a valores no negativos y normalizarlos

como una distribución. La confianza se obtiene por interpolación lineal entre la entropía de una distribución concentrada en una acción (confianza 1) y la de una distribución uniforme (confianza 0), lo que equivale a $1 - H_{\text{norm}}$ (Ecuación (2.16)) sobre esa distribución derivada. La salida se registra junto con la acción seleccionada y un tiempo de respuesta simulado. Las trazas de la Figura 4.2 permiten inspeccionar conjuntamente la confianza, la acción y el desempeño por secuencia.

Dos limitaciones deben quedar explícitas. Primero, la distribución sobre la que se calcula la entropía se deriva de valores Q y no de probabilidades aprendidas, por lo que la confianza es un indicador heurístico y no una probabilidad calibrada. Segundo, el material disponible no incluye una prueba que relacione la confianza con la tasa de acierto ni con el desempeño por secuencia; establecer esa relación requeriría un análisis adicional. La comparación con la confianza humana se mantiene, en consecuencia, como analogía conceptual: las referencias neurocognitivas motivan la interpretación, pero los experimentos no miden señales fisiológicas. Las implicaciones se discuten en el Cap. 6.

6. Discusión

Los resultados del Cap. 5 respaldan la hipótesis H1a: `pes_trf` es el mejor modelo individual y conserva su rendimiento bajo el conjunto de perturbaciones evaluado. La hipótesis H1b, que propone $1 - H_{\text{norm}}$ como indicador de confianza de cada agente y como criterio de ponderación en la decisión conjunta de los ensambles, queda respaldada en el plano algorítmico: la medida es reproducible y los seis ensambles la utilizan, pero el diseño no aísla su contribución y no se dispone de una validación con participantes humanos ni de medidas fisiológicas. Esta sección discute primero los modelos individuales, luego los ensambles y el papel de la confianza en la decisión conjunta, y finalmente los escenarios de extrapolación y el caso de `pes_a2c`.

6.1. Modelos individuales

Las dos familias individuales se separan por el tipo de perturbación que más las afecta. Los agentes tabulares optimizados (`pes_q1`, `pes_dq1`) tienen su peor celda en `sev_extrapolate_high` (0,762 y 0,785) y degradación media positiva (+0,015 y +0,019):

la tabla Q tiene 10 niveles de severidad (0–9) y la inferencia recorta el índice de estado al último nivel, de modo que el agente responde a $S \in \{10, 11, 12\}$ con la política aprendida para $S = 9$. Los agentes profundos densos y actor–crítico (**pes_dqn**, **pes_a2c**, **pes_trf**) tienen su peor celda en **len_extrapolate_long** (0,841, 0,826 y 0,860) y degradación media negativa (−0,005, −0,009 y −0,002): reciben $S/9 > 1$ como entrada continua y mantienen su rendimiento en severidad, pero las secuencias más largas que las de entrenamiento exigen administrar el presupuesto durante más pasos de los que la política aprendió a anticipar. **pes_rdn** es el caso intermedio: su peor celda está en severidad (0,831) y su degradación media es positiva (+0,010), a pesar de recibir la misma ventana de $W = 6$ estados que el Transformer.

En la condición de referencia las medias de las seis arquitecturas se sitúan entre 0,887 y 0,899, con la excepción de **pes_trf** (0,927). La diferencia del Transformer frente a **pes_q1** ($d = 0,78$), **pes_rdn** ($d = 0,57$) y **pes_dqn** ($d = 0,51$) es de tamaño medio en los tres contrastes reportados (Tabla 5.2). El ordenamiento cambia poco bajo perturbación: **pes_trf** conserva la mayor media y la menor dispersión, y las familias mantienen sus puntos débiles respectivos.

6.1.1. Interpretación del rendimiento del Transformer

La dinámica de severidad (Ecuación 4.2) es acumulativa: un déficit de recursos en un paso no produce un costo inmediato proporcional, sino una amplificación geométrica sobre los pasos siguientes por el factor $\beta = 1,4$. Una política que sólo observa el estado instantáneo (R, t, S) dispone de menos información sobre la evolución reciente que una que recibe una ventana de estados pasados.

El codificador Transformer, al procesar la ventana de $W = 6$ estados normalizados con atención causal (Ecuación 2.11), puede construir una representación sensible a la tendencia de la severidad. Esta interpretación es compatible con el mejor desempeño observado, pero el diseño experimental no la demuestra: el modelo recurrente recibe la misma ventana y obtiene un resultado inferior, y no se realizó un estudio de ablación que elimine o permute la información histórica. Tampoco puede descartarse que parte de la diferencia provenga de los hiperparámetros, que se optimizaron para **pes_dqn** y se reutilizaron en las variantes recurrente y Transformer.

6.2. Ensamblares y confianza en la decisi3n conjunta

Los seis ensamblares comparten los miembros `pes_dqn`, `pes_rdqn` y `pes_trf` —las cinco variantes optimizables a~anaden el actor de `pes_a2c` con peso reducido— y todos ponderan a cada miembro por su confianza $c_k = 1 - H_{\text{norm}}(p_k)$ (Sec. 4). Lo que cambia entre ellos es la regla que convierte esas cantidades en una acci3n. Esto permite leer la suite de ensamblares como una comparaci3n entre reglas de decisi3n conjunta sobre un mismo conjunto de opiniones ponderadas por confianza.

La Tabla 5.3 muestra que `pes_ens` obtiene la mejor media bajo perturbaci3n de la suite y la menor degradaci3n m3xima. Su referencia (0,937) supera a la del Transformer individual (0,927), alcanza el valor normalizado m3ximo (1,000) en `sev_extrapolate_high` y `joint_extrap_both`, y su peor celda (`len_extrapolate_long`, 0,900) queda 0,040 por encima de la del Transformer en el mismo escenario (0,860). El resultado es descriptivo del benchmark y admite dos precauciones:

1. La regla de combinaci3n de `pes_ens` incluye, adem3s del promedio ponderado por confianza, una mezcla con una gaussiana centrada en la severidad actual y una cota de seguridad para severidades altas (Sec. 4). No es posible atribuir la ganancia a la ponderaci3n por confianza ni al promedio de modelos por s3 solos sin un estudio de ablaci3n de esas heur3sticas.
2. Cada modelo se compara con su propia condici3n de referencia y el benchmark utiliza una 3nica semilla; por ello no se afirma una robustez general ni se extrapolan los resultados a un despliegue operativo.

Las cinco variantes restantes matizan el papel de la confianza en la decisi3n conjunta. `pes_ens_trf_guard` reproduce casi exactamente al Transformer individual: referencia 0,928 frente a 0,927, y celdas id3nticas hasta la tercera cifra en `sev_extrapolate_high` (0,996) y `joint_extrap_both` (0,997). Esto indica que la compuerta log3stica sobre c_{trf} (Ecuaci3n (4.13)) se abre en la gran mayor3a de las decisiones, de modo que el respaldo ponderado rara vez interviene; el contraste `pes_ens` frente a `pes_ens_trf_guard` tiene $d = 0,17$, mientras que frente a `pes_ens_consensus` alcanza $d = 0,62$ (Tabla 5.4). `pes_ens_sprb` y `pes_ens_accq`, que aplican la misma ponderaci3n $\tilde{w}_k = w_k c_k$ pero promedian o votan sin priorizar a ning3n miembro, obtienen referencias iguales (0,914)

e inferiores al Transformer solo, y su peor celda está en `sev_extrapolate_high` (0,860 y 0,859), precisamente donde el Transformer alcanza 0,996 y los otros dos miembros 0,890 y 0,831. En ese escenario la ponderación por confianza con $\rho = 1$ no basta para que el miembro más acertado domine la decisión conjunta. `pes_ens_consensus` parte de la referencia más baja de la suite (0,893) y presenta la mayor mejora media bajo perturbación ($-0,010$), con su peor celda en la extrapolación de longitud (0,831); `pes_ens_consensus_prior` eleva la referencia a 0,918 y el peor caso a 0,876, lo que describe el efecto conjunto del consenso y del prior de severidad en este banco de pruebas. Ninguna de estas comparaciones aísla la ponderación por confianza: las variantes difieren en la regla completa, no sólo en ρ .

En síntesis, el Transformer es el mejor modelo individual y `pes_ens` el mejor ensamble y el mejor sistema del benchmark. La confianza por entropía es un componente operativo de las seis reglas de decisión conjunta, pero los resultados muestran que su efecto depende de la regla en la que se inserta: mejora el peor caso cuando acompaña a una priorización explícita del miembro más fiable o a un prior de severidad, y no evita la pérdida frente al mejor miembro cuando se usa como único criterio de promediado. Esta distinción evita convertir la ventaja del Transformer individual en una afirmación sobre todos los sistemas evaluados. Los heatmaps estadísticos de la suite de ensambles muestran estas diferencias como cambios de media, tamaños de efecto y desplazamientos de la política; no constituyen una validación independiente de la regla de combinación.

6.3. Entropía como indicador de confianza algorítmica

La medida $1 - H_{\text{norm}}$ es reproducible e interpretable: toma valores cercanos a 1 cuando la distribución derivada de las salidas de un agente se concentra en una acción y cercanos a 0 cuando se reparte entre varias. Registrada por secuencia para `pes_trf` y calculada por paso para cada miembro de los ensambles, cumple la función de un indicador operativo. No demuestra, sin embargo, que el agente posea una noción de confianza comparable a la humana ni que la medida sea predictiva del acierto: esa relación no se evaluó en este trabajo, y los artefactos del benchmark no almacenan los valores c_k por paso que permitirían estudiarla en los ensambles.

La relación con Boldt y Yeung [21] y Fleming [22] es, por tanto, una analogía conceptual

con la confianza humana estudiada mediante EEG, y la ponderación por confianza de los ensambles es una analogía algorítmica con la integración de opiniones en las decisiones grupales humano-máquina. La contribución metodológica consiste en trasladar esas preguntas al comportamiento de agentes artificiales mediante una medida de entropía, no en sustituir la medición fisiológica.

6.4. Escenarios de extrapolación

Las tres condiciones de extrapolación mostradas en la Sec. 5.8 permiten inspeccionar por separado cambios de severidad, de longitud y de ambos factores a la vez. Los modelos tabulares son los más afectados por la extrapolación de severidad, ya que sus severidades se recortan al último nivel de la tabla aprendida (Sec. 6.1); los modelos profundos y los ensambles se ven más afectados por la extrapolación de longitud. Las condiciones estructurales modifican el número de bloques o secuencias sin cambiar la regla de generación y reproducen exactamente el valor de referencia; deben interpretarse como una comprobación del protocolo de agregación, no como una prueba de robustez.

6.5. Lectura de los resultados de `pes_a2c`

`pes_a2c` presenta la menor media individual en la referencia (0,887) pero una media bajo perturbación superior (0,896), con resultados altos en los escenarios conjuntos. Su lectura debe hacerse junto con las matrices de degradación y de divergencia de acciones, que permiten estudiar si el cambio de escenario altera la política. Esas matrices no autorizan, por sí solas, a diagnosticar una insensibilidad del actor al contexto; ese diagnóstico requeriría inspeccionar las distribuciones de acciones por escenario y una prueba específica.

7. Conclusiones

Esta tesis presentó **mPES**, una plataforma de aprendizaje por refuerzo para la asignación óptima de recursos en una pandemia simulada, y la utilizó como banco de prueba para una comparación rigurosa entre seis modelos individuales y seis variantes de ensamble. La suite individual incluye `pes_q1`, `pes_dql`, `pes_dqn`, `pes_rdqn`, `pes_a2c` y `pes_trf`; la suite de ensambles incluye `pes_ens`, `pes_ens_sprb`, `pes_ens_accq`, `pes_ens_consensus`,

`pes_ens_consensus_prior` y `pes_ens_trf_guard`.

7.1. Contraste de las hipótesis

H1a (arquitectónica): respaldada. `pes_trf` obtiene la mayor media de la suite individual en la condición de referencia (0,927) y en los 21 escenarios de perturbación (0,930), con degradación media negativa (−0,002) y la menor dispersión entre secuencias. La diferencia frente a `pes_q1` tiene $d = 0,78$ y frente a `pes_rdqn` $d = 0,57$ (Tabla 5.2). Su peor escenario es `len_extrapolate_long` (0,860), de modo que la conservación del rendimiento es clara en severidad y más limitada en longitud. El respaldo se circunscribe a una semilla y a los hiperparámetros reutilizados de `pes_dqn`.

H1b (confianza y decisión conjunta): respaldada parcialmente. La medida $1 - H_{\text{norm}}$ se calcula de forma reproducible para cada agente y se comporta como indicador operativo: crece cuando la distribución derivada de los valores de acción se concentra en una opción. Como criterio de ponderación en la decisión conjunta, los seis ensambles la utilizan y el sistema con mejor rendimiento del benchmark, `pes_ens`, modula sus pesos con ella; sin embargo, `pes_ens_sprb` y `pes_ens_accq`, que aplican la misma ponderación con una regla de combinación distinta, quedan por debajo del Transformer individual. La ponderación por confianza es, por tanto, un componente utilizable pero no suficiente por sí solo, y su contribución aislada no se midió mediante ablación. No se realizó una calibración frente al acierto ni un estudio con participantes; la analogía con la decisión conjunta humano-máquina queda como motivación y línea de trabajo futuro.

7.2. Conclusiones principales

1. **El Transformer es el mejor modelo individual del benchmark comparativo.** En la condición de referencia alcanza $\bar{r} = 0,927$ y presenta la mayor media individual bajo perturbación, 0,930, con la menor dispersión entre secuencias. Frente a la línea de base tabular optimizada, la diferencia tiene un tamaño de efecto $d = 0,78$.
2. **Los ensambles no superan al Transformer de forma uniforme.** `pes_ens`

obtiene la mejor media bajo perturbación del benchmark (0,939) y la menor degradación máxima; otras variantes quedan por debajo del Transformer individual. La ventaja de **pes_ens** no puede atribuirse sólo al promedio de modelos, dado que su regla de combinación incluye heurísticas adicionales.

3. **La entropía proporciona un indicador de confianza interpretable y utilizable en la decisión conjunta.** La medida $1 - H_{\text{norm}}$ acompaña a cada acción del Transformer y pondera a los miembros de los seis ensambles. Su relación con la confianza humana es una analogía motivada por la literatura; no se realizó una validación fisiológica ni una calibración probabilística.
4. **El benchmark es reproducible.** El módulo `general/scripts/benchmark.py`, seguido por el análisis y la generación de figuras, organiza dos suites de 132 celdas cada una, con 64 secuencias por celda y una semilla fija.

7.3. Limitaciones del estudio

Las conclusiones deben interpretarse considerando las limitaciones del diseño experimental. En primer lugar, el benchmark se ejecutó con una *semilla* fija (42) por celda. Aunque las $n = 64$ secuencias por escenario permiten estimar diferencias entre arquitecturas, una réplica con varias semillas sería necesaria para separar con mayor rigor el efecto algorítmico de la variabilidad aleatoria, especialmente en los extremos del ranking.

La limitación principal proviene del propio escenario utilizado como representante de una decisión secuencial compleja. El entorno *Pandemic Scenario* modela de forma controlada una dinámica acumulativa: cada asignación modifica la severidad futura y obliga a administrar un presupuesto limitado a lo largo de una secuencia. En ese sentido, constituye un banco de pruebas adecuado para estudiar anticipación, memoria y robustez ante cambios en las condiciones de entrada. Sin embargo, sigue siendo una representación simplificada: no reproduce la totalidad de las variables, restricciones y retroalimentaciones presentes en una decisión del mundo real.

Por ello, los resultados identifican un ajuste entre arquitectura y estructura del problema, no una superioridad universal de un modelo. Además, la complejidad comportamental de la decisión humana incluye factores como preferencias, interpretación de evidencia,

presión social, confianza y deliberación, que no están representados en la política del agente. La relación entre la entropía algorítmica y la confianza humana debe entenderse, por tanto, como una analogía funcional y como una limitación del propio hallazgo, no como una equivalencia neurofisiológica. La generalización a otros dominios requiere nuevos escenarios y estudios con participación humana.

7.4. Trabajo futuro

- **Réplica con varias semillas.** Repetir el barrido de las dos suites de 6×22 celdas con semillas independientes permitiría construir intervalos de confianza alrededor del ordenamiento de la Tabla 5.1 y verificar la estabilidad de los resultados en los escenarios de extrapolación (Sec. 6.4).
- **Estudios de ablación.** Permutar o eliminar la ventana de estados del Transformer y del modelo recurrente, desactivar las heurísticas de `pes_ens` y fijar $\rho = 0$ en las variantes de ensamble, para aislar la contribución de cada componente — incluida la ponderación por confianza — al rendimiento observado.
- **Validación de la medida de confianza.** Relacionar $1 - H_{\text{norm}}$ con el desempeño por secuencia mediante curvas de calibración, y extender el cálculo a los demás modelos con un procedimiento de calibración común.
- **Sustituir el escenario simulado** por series temporales reales (por ejemplo, datos de COVID-19 desagregados por región) y evaluar la generalización frente a los marcos de modelado epidemiológico [27].
- **Acoplamiento humano–agente.** Diseñar interfaces que comuniquen $1 - H_{\text{norm}}$ al decisor humano y medir su efecto en la calidad de la decisión conjunta, en la línea de los estudios de equipos humano–máquina [29].
- **Pesos dependientes del contexto en el ensamble.** Los pesos w_k de la Ecuación (4.7) son constantes fijadas en la configuración; una extensión es aprender pesos $w_k(s)$ dependientes del estado.

7.5. Conclusión general

En el escenario controlado de mPES, la arquitectura Transformer obtiene el mejor resultado entre los modelos individuales, y el benchmark permite comparar ese comportamiento con seis ensambles bajo un mismo protocolo. La medida de confianza basada en entropía ofrece una señal interpretable para inspección, pero su valor como estimador de acierto requiere calibración y validación posteriores. Estas conclusiones se limitan al entorno, los artefactos y la semilla descritos.

8. Agradecimientos

A mi director, **Dr. Rodrigo Ramele**, por la guía paciente y rigurosa a lo largo de la Maestría en Ciencia de Datos, esta tesis es deudora directa de su visión sobre el futuro de la tecnología.

A mi esposa, **Analía Giménez**, por su apoyo durante cada etapa de mi vida profesional y cada uno de los proyectos de nuestra vida juntos.

A mis **padres**, que me dieron la oportunidad de formarme como ingeniero y, con ello, los cimientos para hoy acceder a una Maestría. Su esfuerzo silencioso está en el origen de cada uno de mis proyectos.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic experiment scenario. <https://github.com/BCI-NE/PES>, 2022. Financiado por Dstl/UK MoD y US/UK DoD BARI. Accedido: mayo 2026.
- [2] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, K. G. Arjun, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032, 2024. URL <https://arxiv.org/abs/2407.17032>. Aceptado en NeurIPS Datasets and Benchmarks 2025.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [4] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [5] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [6] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, 1999.
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [8] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- [9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [10] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.
- [11] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [13] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [14] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [15] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods

- for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.
- [16] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations (ICLR)*, 2021.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [18] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [19] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [20] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [21] Annika Boldt and Nick Yeung. Shared neural markers of decision confidence and error detection. *Journal of Neuroscience*, 35(8):3478–3484, 2015.
- [22] Stephen M. Fleming. Metacognition and confidence: A review and synthesis. *Annual Review of Psychology*, 75:241–268, 2024.
- [23] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [24] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.
- [25] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [26] Bernard L. Welch. The generalization of Student’s problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [27] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021.
- [28] BCI-NE Lab. OpenNeuro Dataset ds004477, 2022. URL https://nemar.org/dataexplorer/detail?dataset_id=ds004477. Disponible en NEMAR/OpenNeuro.

- [29] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [30] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.
- [31] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [32] Charles Packer, Katelyn Gao, Jernej Kos, Philipp Krähenbühl, Vladlen Koltun, and Dawn Song. Assessing generalization in deep reinforcement learning. *arXiv:1810.12282*, 2018.
- [33] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [34] Maximiliano Vega. mPES: Multiple pandemic experiment suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: mayo 2026.
- [35] Vincy Fon and Francesco Parisi. Litigation and the evolution of legal remedies: A dynamic model. *Public Choice*, 116(3–4):419–433, 2003. doi: 10.1023/A:1024822710849.

Apéndice

A. Comandos de reproducción

La Tabla A.1 resume los comandos canónicos que regeneran cada uno de los artefactos reportados en esta tesis. Todos se ejecutan desde la raíz del *workspace* mPES con el entorno virtual activado (`win_mpes_env` en Windows), Python 3.12 y las dependencias fijadas en `utils/config/requirements.txt`.

Cuadro A.1: Comandos para reentrenar los modelos, optimizar hiperparámetros y ejecutar la evaluación del benchmark en cada paquete del repositorio mPES.

Paquete	Entrenamiento	Optimización bayesiana
<code>pes_base</code>	<code>python -m tabular.pes_base</code>	—
<code>pes_q1</code>	<code>python -m tabular.pes_q1</code>	<code>python -m tabular.pes_q1.ext.optimize_rl</code>
<code>pes_dq1</code>	<code>python -m tabular.pes_dq1</code>	<code>python -m tabular.pes_dq1.ext.optimize_rl</code>
<code>pes_dqn</code>	<code>python -m ml.pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>
<code>pes_rdqn</code>	<code>python -m ml.pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>
<code>pes_a2c</code>	<code>python -m ml.pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>
<code>pes_trf</code>	<code>python -m ml.pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>
<code>pes_ens</code>	<code>python -m ens.pes_ens</code>	— (sin parámetros entrenables)
<code>pes_ens_sprb</code> , <code>pes_ens_accq</code>	inferencia en <code>h1/ens</code>	—
<code>pes_ens_consensus</code> , <code>pes_ens_consensus_prior</code>	inferencia en <code>h1/ens</code>	—
<code>pes_ens_trf_guard</code>	inferencia en <code>h1/ens</code>	—

B. Procedimiento de ejecución del benchmark

El benchmark se ejecuta por suite con el comando reanudable:

```
python -m general.scripts.benchmark run --suite both
```

```
python -m general.scripts.analysis
```

```
python -m general.scripts.figures
```

El procedimiento:

1. ejecuta cada combinación (modelo, escenario) de las suites `individual` y `ensemble`, reutilizando los artefactos entrenados de cada paquete y sustituyendo temporalmente sus archivos de entrada por los del escenario;
2. agrega el rendimiento por secuencia con `general/scripts/analysis.py` para producir las matrices `global_mean`, `std`, `min`, `max`, `stress_degradation`, `welch_p`, `welch_logp`, `cohen_d` y `action_kl` en `general/results/<suite>/matrices/`;

3. genera las figuras y los contrastes pareados con `general/scripts/figures.py`;
4. emite el resumen `general/results/<suite>/report.md`.

C. Escenarios de extrapolación

Los tres escenarios de extrapolación utilizados en las curvas de la Sec. 6.4 son:

- `sev_extrapolate_high`: severidades por encima del rango de la condición de referencia.
- `len_extrapolate_long`: secuencias más largas que las observadas durante entrenamiento.
- `joint_extrap_both`: combinación simultánea de las dos anteriores.

Las tres columnas estructurales (`struct_more_total`, `struct_many_short_blocks`, `struct_few_long_blocks`) reproducen el valor de la condición de referencia, con degradación nula, y sirven como comprobación de que la agregación no depende de la partición en bloques.